

Integrantes: _____

Objetivo: El alumno aprenderá a elaborar circuitos usando compuertas lógicas, entenderá conceptos como tabla de la verdad, tierra digital y tendrá una breve introducción a las funciones booleanas.

Marco teórico

Lógica Binaria

La lógica binaria trata con variables que toman dos valores discretos y con operaciones que tienen significado lógico.

Los dos valores que toman las variables pueden designarse con nombres diferentes:

- a) Verdadero y falso
- b) Si y no

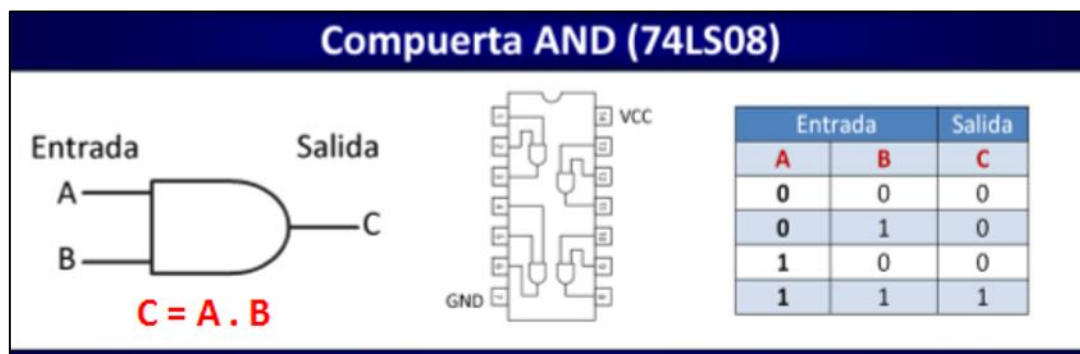
Sin embargo, para nosotros será conveniente pensar en términos de bits y asignarles los valores de **1** y **0**.

La lógica es equivalente a un algebra llamada booleana. La lógica binaria consta de variables binarias y operaciones lógicas. Las variables se denotan con letras del alfabeto como: **A, B, C, x, y, z**, etc. y cada variable tiene dos y sólo dos valores distintos: 1 y 0.

Operaciones básicas de la lógica binaria

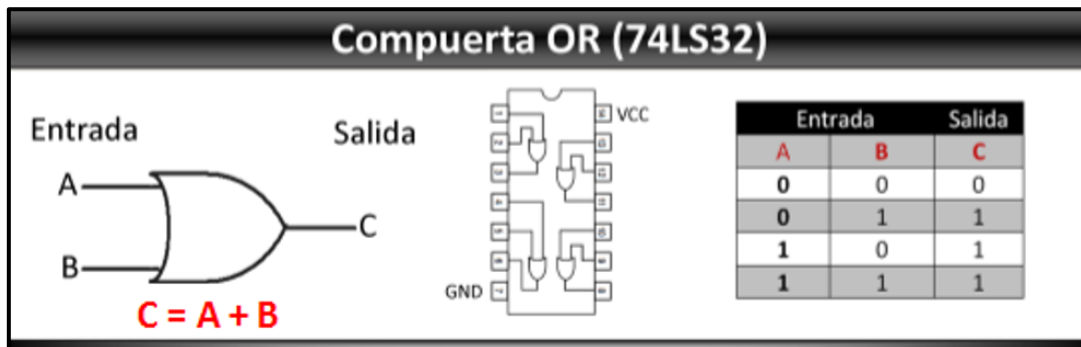
Hay tres operaciones lógicas básicas: AND, OR y NOT.

AND (Y): Esta operación se representa mediante un punto o por la ausencia de operador. La compuerta AND representa la operación de multiplicación. En la *figura 1.1* se puede observar su símbolo, su función característica y su tabla de la verdad.

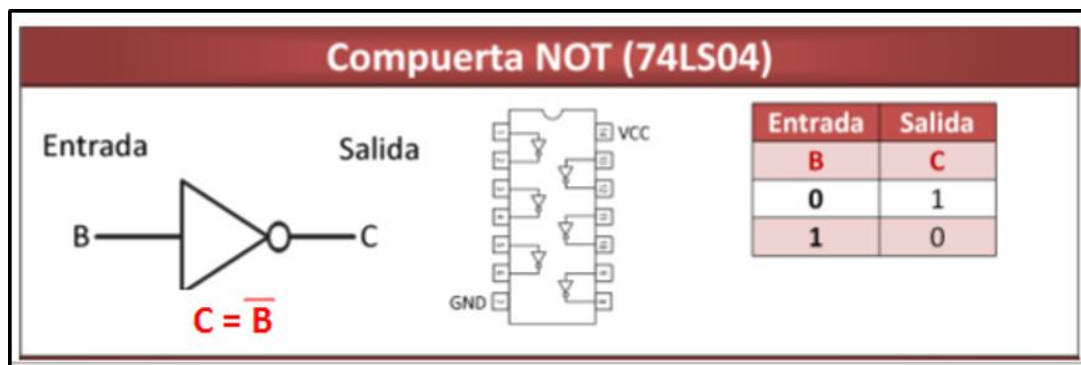


Práctica #1 Compuertas Lógicas

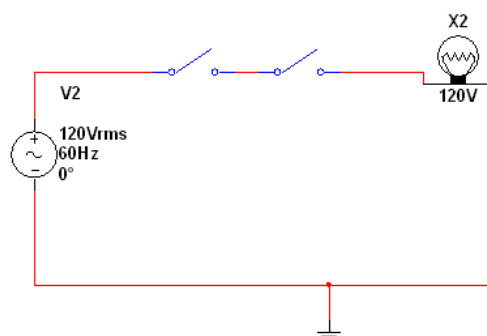
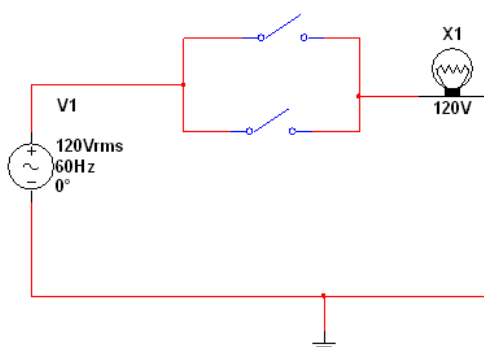
OR (O): Esta operación se representa mediante un signo de suma (operación de suma). Por ejemplo $A + B = C$.



NOT (NO): Esta operación se representa por una sola comilla (algunas veces por una barra). Representa la negación. Se le conoce como compuerta complemento.

Circuitos Lógicos

Los circuitos digitales también se denominan circuitos lógicos ya que, con la entrada apropiada, establecen trayectorias lógicas de manipulación. Cualquier información que se desee para computación o control puede operarse por el paso de señales binarias a través de diversas combinaciones de circuitos lógicos, cada señal representa una variable y lleva un bit de información. Por ejemplo en las figuras se puede observar la representación de las compuertas OR y AND con interruptores..



Práctica #1 Compuertas Lógicas**Tierra Digital**

A menudo cuando se trabaja con circuitos digitales algunos símbolos puede ser un tanto diferentes a cuando se trabaja con circuitos analógicos. Por ejemplo en la tabla 1.1 se pueden observar los tres tipos de símbolos para la conexión a tierra más comunes. En este manual será vera frecuentemente el símbolo de “tierra digital”.

Tabla 1.1 Simbología para conexiones a tierra.

Simbología de tierra.		
	Tierra analógica	Utilizado para referencia de potencial a cero (Tierra analógica) y también para protección contra descargas eléctricas.
	Tierra del chasis.	Conectada al chasis del circuito o también conocida como (tierra flotada). Ejemplo: en un auto la tierra es el chasis del coche.
	Tierra digital	Punto en común con potencial a cero en un circuito digital.

Desarrollo de la práctica:**Materiales:**

- 2 NAND 74LS00
- 2 NOR 74LS02
- 2 NOT 74LS04
- 2 AND 74LS08
- 2 OR 74LS32
- 2 XOR 74LS86
- 2 XNOR 74LS266
- 1 DIPSWITCH DE 8
- 2 RESISTENCIAS DE 470 Ω
- 2 Resistencias de 220 Ω o 330 Ω
- 2 LED'S Rojos
- 2 LED'S Verdes

Práctica 1.1

Construya un circuito eléctrico en su protoboard para comprobar la tabla de la verdad de las compuertas digitales AND, OR, NOT, XOR, NAND, NOR y XNOR.

Por ejemplo:

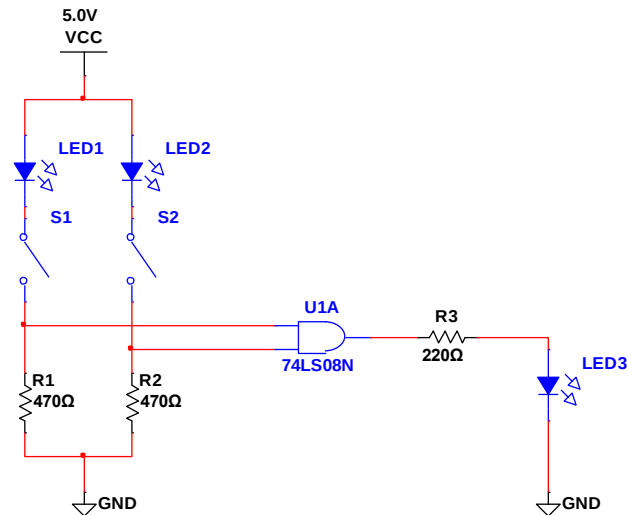


Figura 1. Circuito Verificador de compuertas.

Usa la misma configuración pero variando los diferentes integrados que se muestran en la figura 1.5. Recuerda que si no cuentas con todas las compuertas las puedes obtener a partir de las compuertas básicas AND, OR y NOT.

Compuertas lógicas

Compuerta AND (74LS08)

Entrada Salida

$$C = A \cdot B$$

Entrada A	Entrada B	Salida C
0	0	0
0	1	0
1	0	0
1	1	1

Compuerta NAND (74LS00)

Entrada Salida

$$C = \overline{A \cdot B}$$

Entrada A	Entrada B	Salida C
0	0	1
0	1	1
1	0	1
1	1	0

Compuerta OR (74LS32)

Entrada Salida

$$C = A + B$$

Entrada A	Entrada B	Salida C
0	0	0
0	1	1
1	0	1
1	1	1

Compuerta NOR (74LS02)

Entrada Salida

$$C = \overline{A + B}$$

Entrada A	Entrada B	Salida C
0	0	1
0	1	0
1	0	0
1	1	0

Compuerta XOR (74LS86)

Entrada Salida

$$C = A \cdot \overline{B} + \overline{A} \cdot B$$

Entrada A	Entrada B	Salida C
0	0	0
0	1	1
1	0	1
1	1	0

Compuerta XNOR (74LS266)

Entrada Salida

$$C = A \cdot B + \overline{A} \cdot \overline{B}$$

Entrada A	Entrada B	Salida C
0	0	1
0	1	0
1	0	0
1	1	1

Compuerta NOT (74LS04)

Entrada Salida

$$C = \overline{B}$$

Entrada B	Salida C
0	1
1	0

Práctica 1.2

El consejo directivo de una empresa se encuentra integrado por tres personas. En una de sus juntas se acordó que las votaciones se hicieran de forma secreta; sin embargo, existe el problema de que una persona ajena contará los votos para mantener el secreto del voto. Para evitar el problema se decide hacer lo siguiente:

Se instalará un botón debajo de la mesa de cada directivo. Al centro de la sala de juntas se colocarán dos lámparas, una de color roja y una de color verde. Al momento de votar, si el directivo está a favor presionará el botón, si está en contra no lo presionará. La lámpara color verde deberá encenderse si la mayoría vota a favor. La lámpara de color rojo deberá encenderse si la mayoría está en contra.

- Elabore la tabla de la verdad para obtener la solución a este problema.
- Implemente con compuertas lógicas básicas la solución encontrada.

1.CONCLUSIONES INDIVIDUALES

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Práctica 3.2

1. Con la tabla de la verdad mostrada calcule la función booleana F_1 sin simplificar, usando Minitérminos cuando la combinación de unos y ceros de como resultado un 1 en la función. De igual manera exprese la función en notación abreviada.

Posteriormente simplifique la función obtenida y proceda a implementar el circuito.

2. Con la tabla de la verdad mostrada calcule la función booleana F_2 sin simplificar, usando Maxitérminos, cuando la combinación de 1 y 0 da como resultado un 0 en la función. También exprese la función en notación abreviada.

Posteriormente simplifique la función obtenida y proceda a implementar el circuito.

X	Y	Z	F1	F2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

NOTA: Reporte fotos del circuito así como el procedimiento que se realizó para simplificar las funciones booleanas, de igual manera expresa las funciones en su forma simplificada.

Conclusiones Individuales

Práctica #2 Maxitérminos y Minitérminos

Integrantes: _____

Objetivo: El alumno aplicará sus conocimientos en algebra booleana reduciendo funciones expresadas en Minitérminos y máxterminos.

Marco teórico

El álgebra booleana.

Como cualquier otro sistema matemático deductivo, el álgebra booleana puede definirse como un conjunto de *elementos*, un conjunto de *operadores* y un número de *teoremas* y *postulados*.

Un algebra booleana de dos valores se define en un conjunto de *elementos*:

$$B = \{0,1\}$$

Los operadores usados en el álgebra booleana son:

AND (.) y OR (+)

Además está la operación de negación que muestra el estado inverso de una entrada (*Operación NOT* conocida como complemento).

El álgebra booleana se rige por un conjunto de postulados y teoremas, los más importantes se muestran en la *tabla 1*.

Tabla 1. Teoremas y postulados del algebra booleana.

Postulado 2	a) $X + 0 = X$	b) $X \cdot 1 = X$
Postulado 5	a) $X + X' = 1$	b) $X \cdot X' = 0$
Teorema 1	a) $X + X = X$	b) $X \cdot X = X$
Teorema 2	a) $X + 1 = 1$	b) $X \cdot 0 = 0$
Teorema 3, Involución	$(X')' = X$	
Postulado 3, conmutativo	a) $X + Y = Y + X$	b) $X Y = Y X$
Teorema 4, asociativo	a) $X + (Y + Z) = (X + Y) + Z$	b) $X(YZ) = (XY)Z$
Postulado 4, distributivo	a) $X(Y + Z) = XY + XZ$	b) $X + YZ = (X + Y)(X + Z)$
Teorema 5, de De Morgan	$(X + Y)' = X'Y'$	$(XY)' = X' + Y'$
Teorema 6, absorción	$X + XY = X$	$X(X + Y) = X$

Práctica #2 Maxitérminos y Minitérminos

Los postulados se listaron en pares y se designaron en la *parte a)* y *parte b)*. Una puede obtenerse de la otra si los operadores binarios y los elementos identidad se intercambian. Esta propiedad importante del álgebra booleana se le denomina *principio de dualidad*.

Principio de dualidad: Establece que toda expresión algebraica que pueda deducirse de los postulados del álgebra y seguirá siendo válida si se intercambian los operadores y los elementos identidad.

Maxitérminos Y Minitérminos

Una función booleana puede expresarse como una suma de productos (Minitérminos) o como un producto de sumas (Maxitérminos).

Las variables combinadas con un operador AND se le denomina *Minitérmino* o *producto estándar*. Cada variable es vuelta prima si el bit correspondiente del número binario es cero y no prima si es 1.

Las variables combinadas con un operador OR, con cada variable prima si el bit correspondiente es 1 y no prima si es cero y se le conoce como *Maxitérminos* o *sumas estándar*.

Cada Maxitérmino es su complemento de su Minitérmino y viceversa. Esto se puede observar en la *tabla 2*.

Tabla 2. Minitérminos y Maxitérminos.

			Minitérmino					Maxitérminos	
X	Y	Z	Término	Designación				Término	Designación
0	0	0	$X'Y'Z'$	m_0				$X+Y+Z$	M_0
0	0	1	$X'Y'Z$	m_1				$X+Y+Z'$	M_1
0	1	0	$X'YZ'$	m_2				$X+Y'+Z$	M_2
0	1	1	$X'YZ$	m_3				$X+Y'+Z'$	M_3
1	0	0	$XY'Z'$	m_4				$X'+Y+Z$	M_4
1	0	1	$XY'Z$	m_5				$X'+Y+Z'$	M_5
1	1	0	XYZ'	m_6				$X'+Y'+Z$	M_6
1	1	1	XYZ	m_7				$X'+Y'+Z'$	M_7

Práctica #2 Maxitérminos y Minitérminos

Por lo general se acostumbra que los Minitérminos que aparecen en la función, son los Minitérminos que hacen verdadera a la función y se ve reflejada en la tabla de verdad. En otras palabras cuando cada combinación de la tabla de la verdad da un "1" a la salida de la función. Por ejemplo la función " F_1 " de la *tabla 3* expresada en suma de productos es la siguiente:

$$F_1 = X'Y'Z + XY'Z' + XYZ = m_1 + m_4 + m_7$$

De igual manera se puede obtener una función booleana como productos de suma o Maxitérminos usando los Maxitérminos que hacen cero la función. Por ejemplo la función " F_2 " expresada como productos de suma es la siguiente:

$$F_2 = (X+Y+Z)(X+Y+Z')(X+Y'+Z')(X'+Y+Z) = M_0 . M_1 . M_2 . M_4$$

Función de tres variables				
X	Y	Z	Función f_1	Función f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Las funciones booleanas expresadas como una suma de Minitérminos o producto de Maxitérminos se dice que están en su *forma canónica*.

Notación abreviada de las funciones booleanas

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

En ocasiones es preferible expresar la función booleana en su notación abreviada

$$F(x,y,z) = \Sigma(1,3,6,7)$$

El símbolo Σ representa términos OR, por lo que se tratará de Minitérminos

$$F(x,y,z) = \Pi(0,2,4,5)$$

El símbolo Π representa AND por los que se tratará de Maxitérminos

Desarrollo de la práctica:

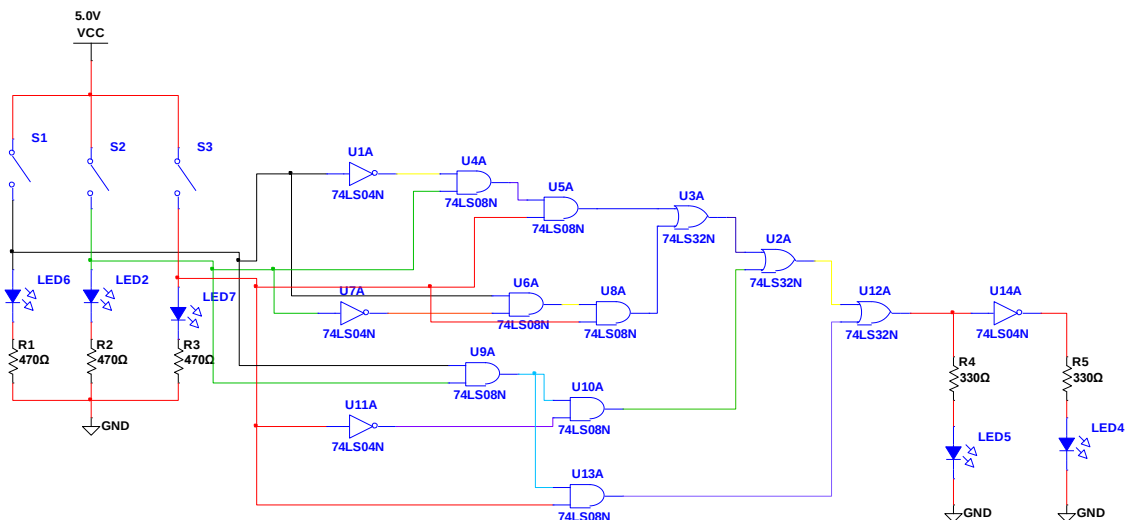
Materiales:

- 2 NAND 74LS00
- 2 NOR 74LS02
- 2 NOT 74LS04
- 2 AND 74LS08
- 2 OR 74LS32
- 2 XOR 74LS86
- 2 XNOR 74LS266
- 1 DIPSWITCH DE 8
- 3 RESISTENCIAS DE 470 Ω
- 2 Resistencias de 220 Ω o 330 Ω
- 3 LED'S Rojos
- 2 LED'S Verdes

Práctica 3.1

Simplifique la función $F = A'BC + AB'C + ABC' + ABC$ expresada como suma de productos (Minitérminos) presentada en la práctica I. El objetivo es implementar el circuito de votaciones de una manera más sencilla. Recuerde que al reducir las ecuaciones booleanas podemos ahorrar compuertas lógicas, lo que favorece a un circuito de menos dimensiones, menos costo y por lo tanto menos complejo.

El circuito sin simplificar se muestra en la figura.



Práctica 3.2

1. Con la tabla de la verdad mostrada calcule la función booleana F_1 sin simplificar, usando Minitérminos cuando la combinación de unos y ceros de como resultado un 1 en la función. De igual manera exprese la función en notación abreviada.

Posteriormente simplifique la función obtenida y proceda a implementar el circuito.

2. Con la tabla de la verdad mostrada calcule la función booleana F_2 sin simplificar, usando Maxitérminos, cuando la combinación de 1 y 0 da como resultado un 0 en la función. También exprese la función en notación abreviada.

Posteriormente simplifique la función obtenida y proceda a implementar el circuito.

X	Y	Z	F1	F2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

NOTA: Reporte fotos del circuito así como el procedimiento que se realizó para simplificar las funciones booleanas, de igual manera expresa las funciones en su forma simplificada.

Conclusiones Individuales

Integrantes: _____

Objetivo: El alumno aplicará sus conocimientos sobre simplificación de funciones booleanas con la ayuda de los mapas de Karnaugh, para implementar un decodificador de BCD a 7 Segmentos con compuertas lógicas.

Marco teórico

Mapas de Karnaugh

Las funciones booleanas pueden simplificarse por medios algebraicos, sin embargo, el procedimiento de minimización es difícil debido a que carece de reglas específicas para predecir cada paso sucesivo en el proceso de manipulación.

El método de mapas proporciona un procedimiento simple y directo para minimizar las funciones booleanas. Este método puede considerarse ya sea como una forma gráfica de una tabla de verdad o como una extensión del diagrama de Venn.

El método de mapas que Veitch fue el primero en proponer y que modificó ligeramente Karnaugh, también se conoce como el “diagrama de Veitch” o “mapa de Karnaugh”.

Mapas de múltiples variables.

Para aplicar los mapas de Karnaugh se recomienda lo siguiente:

- Minimizar la cantidad de grupos.
- Maximizar la cantidad de celdas en cada grupo.
- Agrupar los “1” para Minitérminos o “0” para Maxitérminos en potencias de 2, es decir grupos de 1,2,4,8,16,32, etc. (El mapa puede doblarse para agrupar celdas).
- Puedo agrupar las celdas que ya se hayan usado, el objetivo es agrupar la mayor cantidad de celdas.
- Se analiza que variable no cambia.

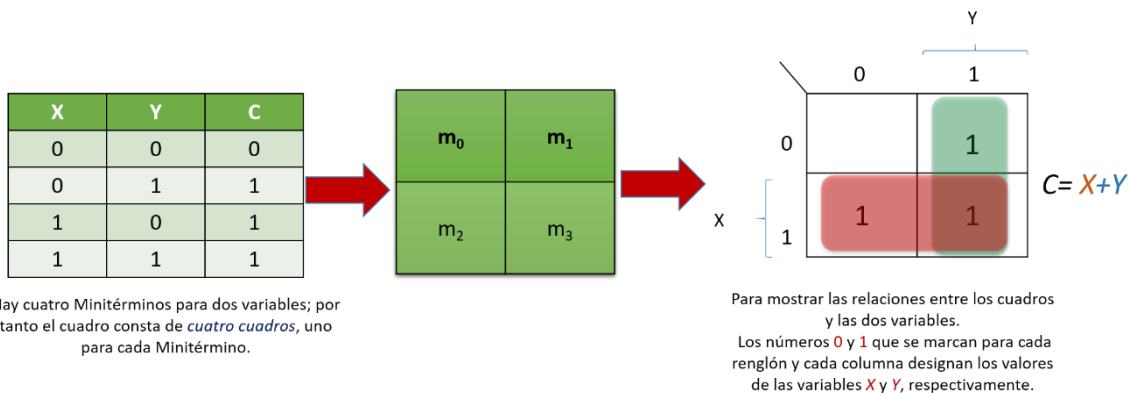
Mapas de 2 variables.

Si se tienen dos variables el mapa tendrá cuatro cuadros, uno para cada Minitérmino. Los números "1" deben acomodarse de manera ordenada como se muestra en la figura.

Luego se agrupan las celdas de acuerdo a las recomendaciones mencionadas anteriormente.

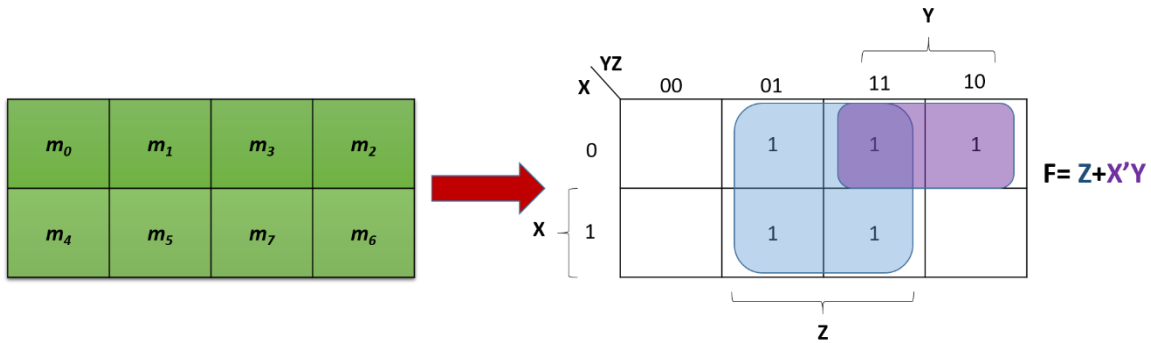
Una vez hechos los grupos se observa en el dominio de que variable se encuentra el grupo. Por ejemplo el grupo de color rosa se encuentra en el dominio de X y también en el de Y y Y' , sin embargo $Y.Y'=0$, por lo tanto el resultado para el grupo de color rojo es X . El grupo de color verde se encuentra en el dominio de Y , de X y X' , similar al anterior $X.X'=0$ dejando para este grupo la variable resultante Y .

Al final se presentan los resultados en forma de suma de productos obteniendo la función booleana $C=X+Y$.

**Mapas de 3 variables.**

En la figura se muestra un mapa de tres variables. Hay 8 Minitérminos para tres variables binarias, por tanto el mapa consta de 8 cuadros. Advierta que los Minitérminos no están acomodados en sucesión binaria, si no en un sucesión similar al código Gray. La característica de esta sucesión es que sólo un bit cambia de valor entre dos columnas adyacentes.

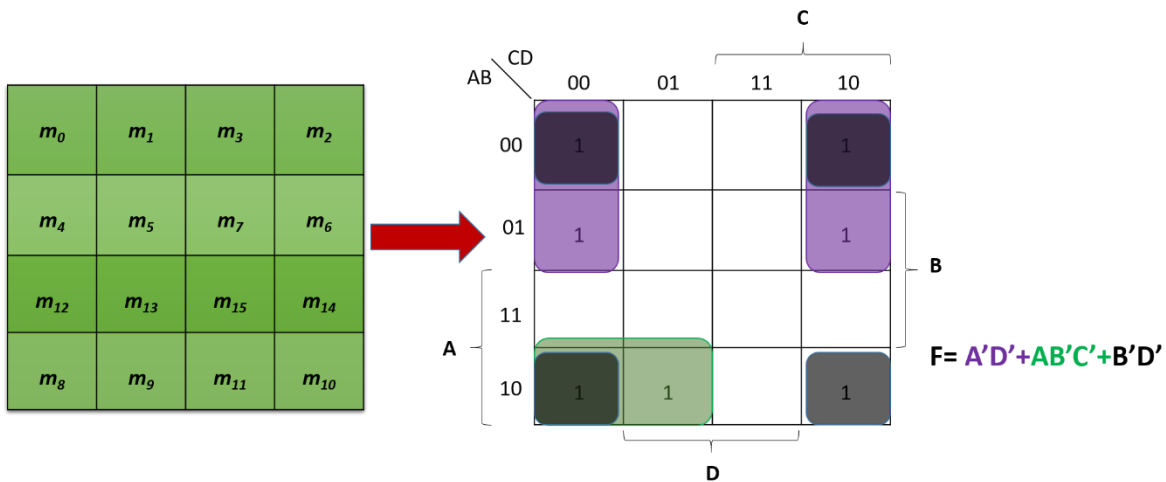
Práctica #3 Mapas de Karnaugh



Las reglas para agrupar los grupos son las mismas. Se puede observar que el conjunto de color azul se encuentra en el dominio de Z y las otras variables X y Y se cancelan. Por lo tanto el grupo de color azul tiene como resultado solamente Z. El grupo de color morado está en el dominio de X', Y y la Z se cancela, dejando los Minitérminos X'Y. La función simplificada es $F = Z + X'Y$.

Mapas de 4 variables.

Un mapa de 4 variables tiene 16 Minitérminos, las filas y columnas se numeran en orden según el código Gray. Como se muestra en la figura.



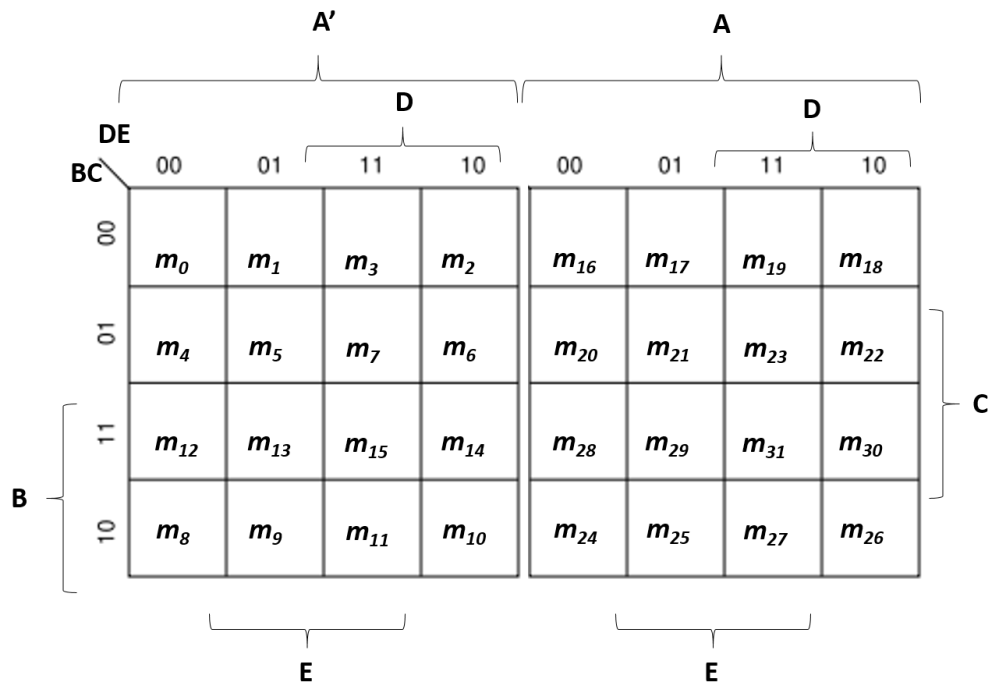
En la figura se puede apreciar que se tienen 3 grupos en el mapa. En el grupo de color verde se obtiene $AB'C'$, del grupo de color morado AD' y el grupo de color negro $B'D'$. Recuerde que este último es válido ya que el mapa de Karnaugh se puede doblar agrupando las cuatro celdas de las esquinas.

Práctica #3 Mapas de Karnaugh

Mapas de 5 variables.

Un mapa para más de cuatro variables no es tan sencillo. Un mapa de 5 variables necesita 32 cuadros y uno de 6 variables 64 cuadros. Cuando hay muchas variables, el número de cuadros aumenta en forma considerable y la geometría para combinar cuadros adyacentes se complica progresivamente.

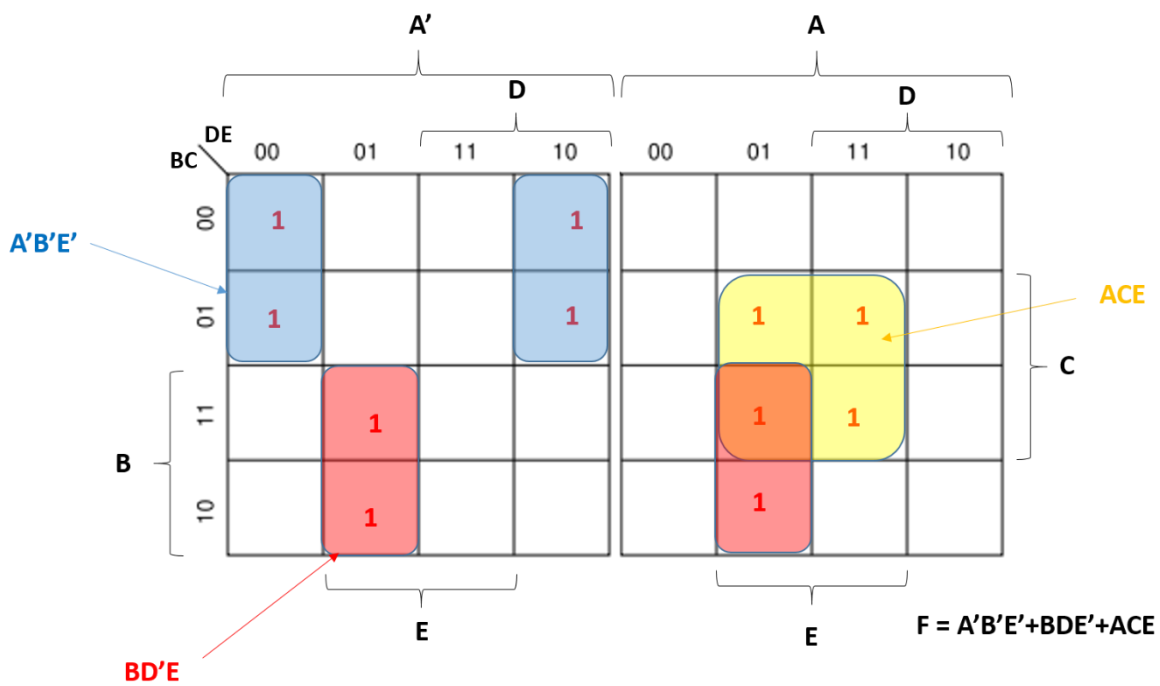
En la figura se muestra un mapa de cinco variables. Este consta de dos mapas de cuatro variables con las variables A, B, C, D y E. La variable A distingue a los dos mapas, como se indica en la parte superior del diagrama. El mapa de cuatro variables de la izquierda representa los 16 cuadros en los que $A = 0$ y los Minitérminos 16 a 31 corresponden a $A = 1$. Los Minitérminos 0 a 15 corresponden a $A = 0$.



Cada mapa de cuatro variables conserva las adyacencias que se definieron anteriormente, cuando se le considera aparte. Además cada cuadro del mapa $A = 0$ es adyacente al cuadro correspondiente del mapa $A = 1$. Por ejemplo el Minitérmino 4 es adyacente con el Minitérmino 20, y el Minitérmino 15, al 31. La mejor forma de visualizar esta nueva regla de adyacencia es imaginar que los dos mapas están uno encima del otro. Cualquiera de los cuadros que queden uno encima del otro se consideran adyacentes.

Práctica #3 Mapas de Karnaugh

Por ejemplo en la figura se muestra la resolución de un mapa de 5 variables, en el cual se tienen tres grupos, el de color azul se ha obtenido al agrupar las celdas de los costados como en un mapa de 4 variables normal y el de color amarillo de igual manera. Sin embargo para el grupo de color rojo hay que aplicar la nueva regla de adyacencia imaginando que se encuentran uno encima del otro, como en ambos mapas los "1" se encuentran en las mismas celdas se forma un grupo de 4. En este último como el grupo se encuentra tanto en A, como en A', esta variable se elimina dejando solamente BD'E.

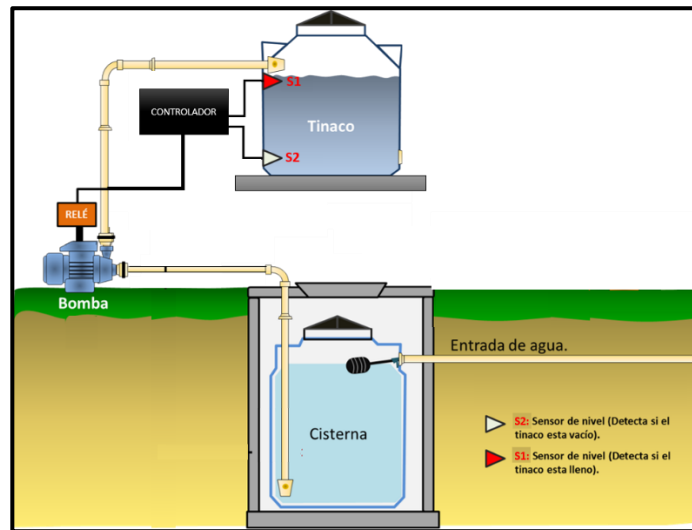


Práctica #3 Mapas de Karnaugh

Condiciones no importa

Las condiciones no importa son aquellas combinaciones de entrada para los que no importa el valor de salida, por ejemplo: porque no se ha especificado el comportamiento del circuito o simplemente son imposibles.

Un ejemplo clásico es un sistema automatizado para el llenado de un tinaco, suponga que se tienen dos sensores en el tinaco, el sensor 1 (S1) indica cuando el tinaco está lleno y el sensor 2 (S2) cuando está vacío. Dependiendo del estado de los sensores el controlador activará la bomba de llenado, como se observa en la *figura*.



Si este comportamiento lo mostramos en una tabla de la verdad quedaría de la siguiente manera.

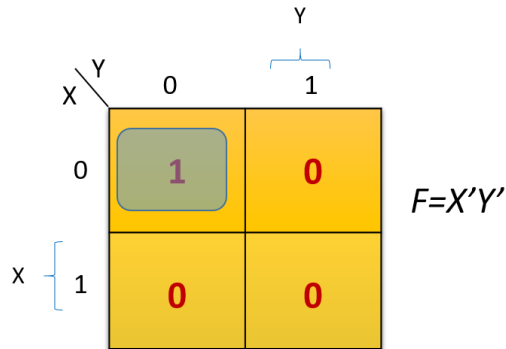
S1	S2	B
0	0	1
0	1	0
1	0	0
1	1	0

Se entiende que los sensores dan una salida de cero cuando no detectan agua y un uno cuando hay agua. Por lo tanto el controlador solo activará la bomba cuando el tinaco este completamente vacío, es decir en la combinación $S1=0$ y $S2=0$. En la tercera combinación ($S1=1$ y $S2=0$) la bomba se mantiene apagada para que se produzca un rango de histéresis y la bomba no se esté encendiendo y apagando en cada momento. La cuarta combinación ($S1=1$ y $S2=1$) indica que el tinaco está lleno, por lo tanto la bomba estará apagada. Sin embargo en la combinación ($S1=0$ y $S2=1$) es una combinación imposible ya que no puede

Práctica #3 Mapas de Karnaugh

estar lleno y vacío al mismo tiempo, en ese caso podríamos mandar un cero, para que la bomba no encienda en esa condición de error.

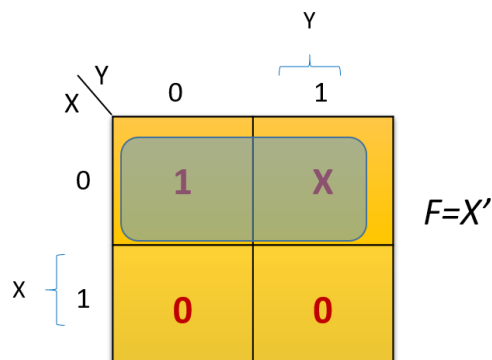
Si lleváramos esto a un mapa de Karnaugh la función resultante quedaría como se muestra en la figura.



Ahora bien como se sabe la combinación (S1=0 y S2=1) es imposible por lo que podemos decir que es una condición no importa y se marca con una X en la tabla de la verdad, como se muestra en la figura.

S1	S2	B
0	0	1
0	1	X
1	0	0
1	1	0

Si esta tabla de la verdad la llevamos de nuevo a un mapa de Karnaugh quedaría como se muestra en la figura.

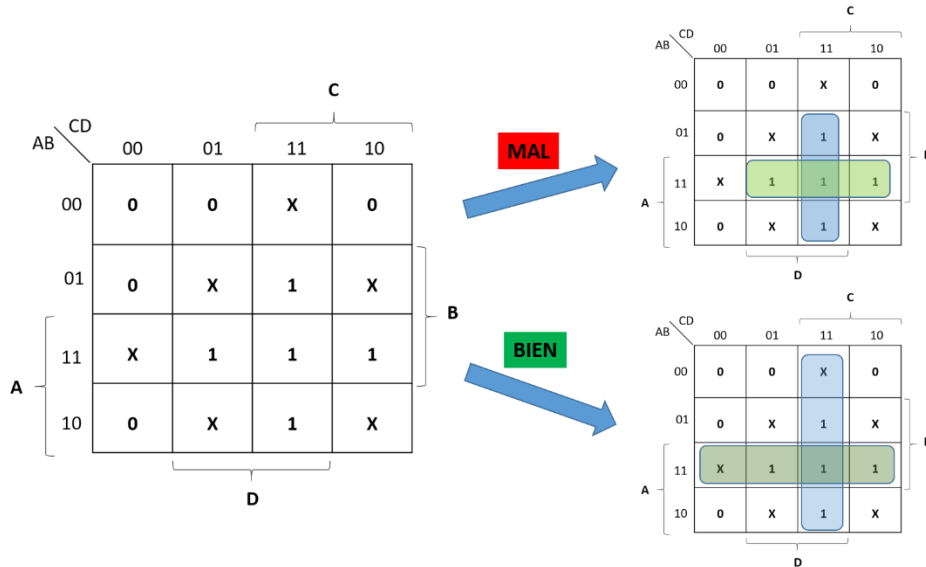


Se puede observar que las X se pueden agrupar con los 1, reduciendo cada vez más la función booleana. Esto ayudará a facilitar mejor la implementación del circuito.

Práctica #3 Mapas de Karnaugh

En las tablas de la verdad y en los mapas de Karnaugh, la salida para las condiciones no importa se indica con la letra X o d. Además estas celdas se toman como si tuvieran valor de 1 o 0, según sea la conveniencia. Para aplicar de manera correcta siga las siguientes indicaciones:

- Usarlas para maximizar el tamaño de los grupos, esto ayudará a disminuir la ecuación, como se observa en la *figura*.
- No agrupar celdas que solamente contengan X.



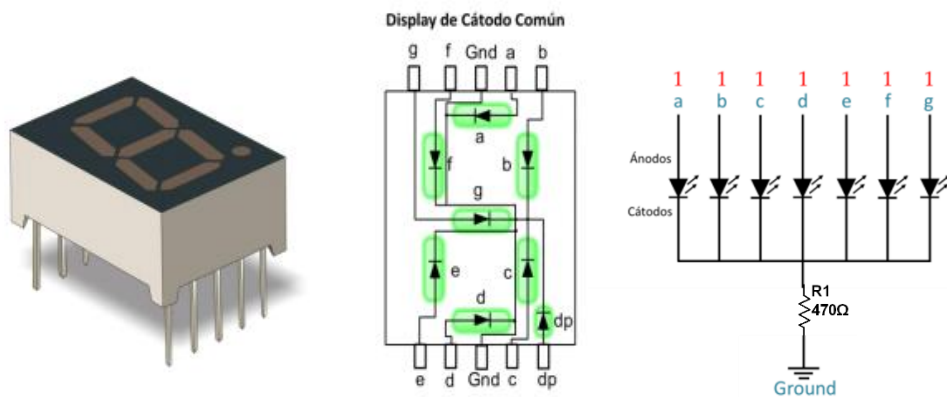
Desarrollo de la práctica:

Materiales:

- 1 Display de 7 segmentos de cátodo común.
- Compuertas NAND 74LS00
- Compuertas NOR 74LS02
- Compuertas NOT 74LS04
- Compuertas AND 74LS08
- Compuertas OR 74LS32
- Compuertas XOR 74LS86
- Compuertas XNOR 74LS266
- 1 DIPSWITCH DE 8
- 1 RESISTENCIAS DE 470 Ω
- 4 Resistencias de 220 Ω o 330 Ω
- 1 Arduino (Opcional).

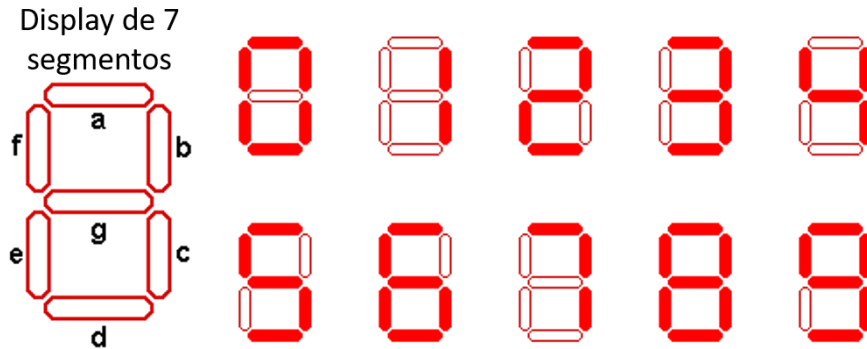
Práctica 4.1- Implementación de un decodificador de BCD a 7 segmentos.

Para implementar el decodificador de BCD a 7 segmentos puede usar un display de cátodo común. En la figura se muestra el diagrama de un display y la forma de conexión.



Práctica #3 Mapas de Karnaugh

Como se observa el display está formado por 7 LED's, los cuales forman los 10 dígitos decimales (0-9) dependiendo del segmento que este encendido. Para encender un segmento del display de cátodo común se envía un 1 lógico. Algunos display agregan un octavo segmento para un punto decimal.



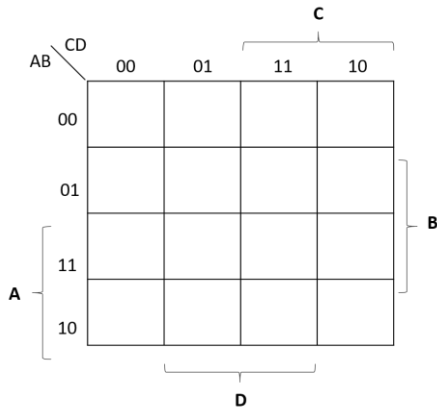
Llene la siguiente tabla de la verdad para formar los 10 dígitos. Recuerde que se requieren 4 bits para los 10 dígitos decimales, por lo tanto se tendrán 16 combinaciones. Sin embargo el BCD sólo llega hasta el número 9, además de que físicamente el display solo puede mostrar hasta el número 9, por lo tanto algunas combinaciones serán condiciones no importa.

ENTRADAS					SALIDAS						
No.	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0							
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0							
3	0	0	1	1							
4	0	1	0	0							
5	0	1	0	1							
6	0	1	1	0							
7	0	1	1	1							
8	1	0	0	0							
9	1	0	0	1							
10	1	0	1	0	x	x	x	x	x	x	x
11	1	0	1	1							
12	1	1	0	0							
13	1	1	0	1							
14	1	1	1	0							
15	1	1	1	1							

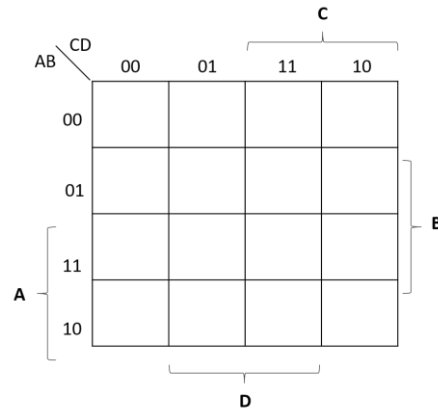
Práctica #3 Mapas de Karnaugh

Procede a reducir con mapas K cada una de las funciones que corresponde a los segmentos del display.

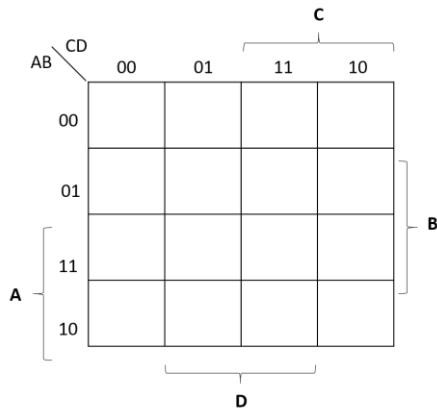
Segmento A. $F_a =$ _____



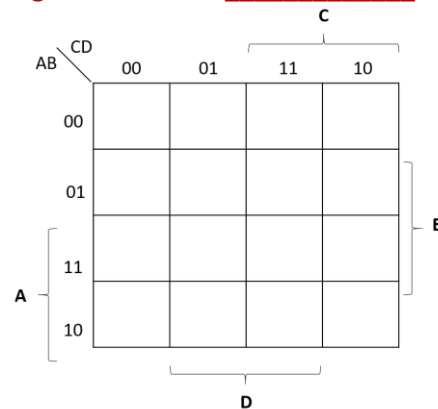
Segmento B. $F_B =$ _____



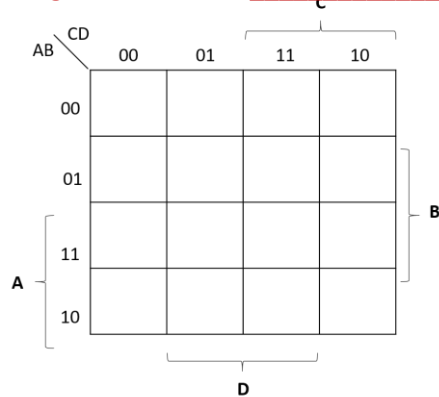
Segmento C. $F_c =$ _____



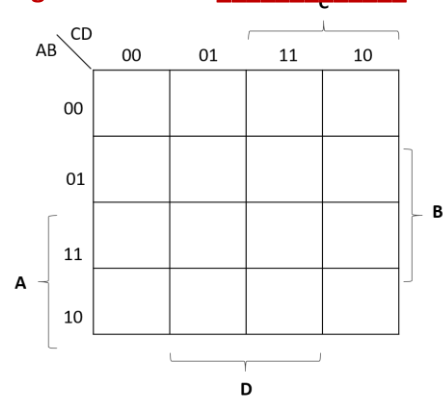
Segmento D. $F_D =$ _____



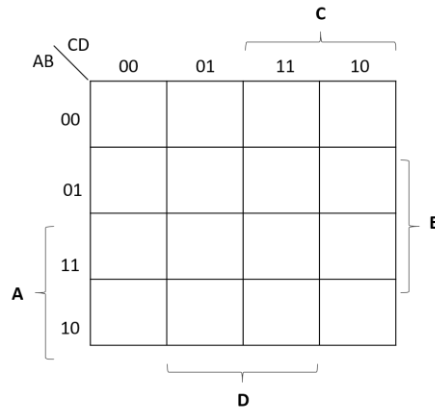
Segmento E. $F_E =$ _____



Segmento F. $F_F =$ _____



Práctica #3 Mapas de Karnaugh

Segmento G. $F_G =$ _____

Con las funciones ya simplificadas implementa el circuito. El circuito físico puede implementarlo usando compuertas básicas o puede hacer uso del Arduino con LabView. Si se quiere realizar con LabView consulte a guía de instalación del LabView y LINX para Arduino en el anexo de este manual.

Reporta el diagrama del circuito y agrega algunas fotos del circuito implementado.

Conclusiones Individuales

Práctica #4 La propiedad universal de las compuertas NAND y NOR.

Integrantes: _____

Objetivo: El alumno demostrará la universalidad de las compuertas NAND y NOR implementando diferentes circuitos lógicos.

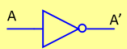
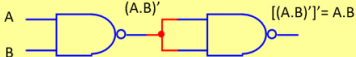
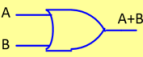
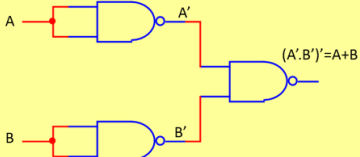
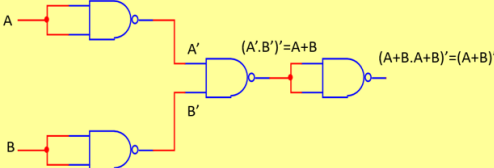
Marco teórico

La propiedad Universal de las compuertas NAND y NOR.

La universalidad de la compuerta *NAND* significa que puede utilizarse como un inversor y que pueden emplearse combinaciones de la compuerta *NAND* para implementar las operaciones *AND*, *OR* y *NOR*. Del mismo modo, la compuerta *NOR* se puede utilizar para implementar el inversor (*NOT*) y las operaciones *AND*, *OR* y *NAND*.

La compuerta NAND como elemento lógico universal

Se dice que la compuerta *NAND* es una compuerta universal porque, cualquier sistema digital puede implementarse con ella. Lo anterior puede demostrarse obteniendo las compuertas *AND*, *OR*, *NOT* y *NOR* a partir de compuertas *NAND*. En la figura se muestra la equivalencia *NAND* de las compuertas básicas.

COMPUERTA BÁSICA	EQUIVALENCIA NAND
	
	
	
	

Práctica #4 La propiedad universal de las compuertas NAND y NOR.**Lógica NAND**

Cualquier función booleana puede llevarse a su equivalente en compuertas NAND. Sin embargo se recomienda llevar la función que se desea implementar a *suma de productos*.

Por ejemplo:

La función F_1 no es producto de sumas por lo que hay que simplificar la función.

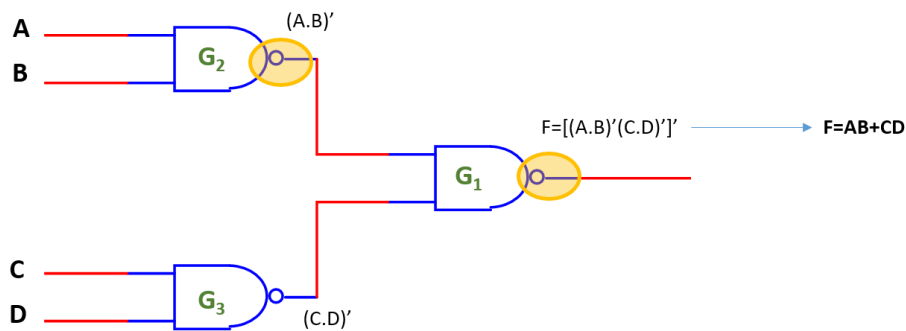
$$F_1 = [(AB)'(CD)']'$$

$$F_1 = [(A'+B')(C'+D')]'$$

$$F_1 = (A'+B')' + (C'+D')'$$

$$F_1 = AB + CD$$

En la siguiente figura se muestra la función implementada con compuertas NAND.

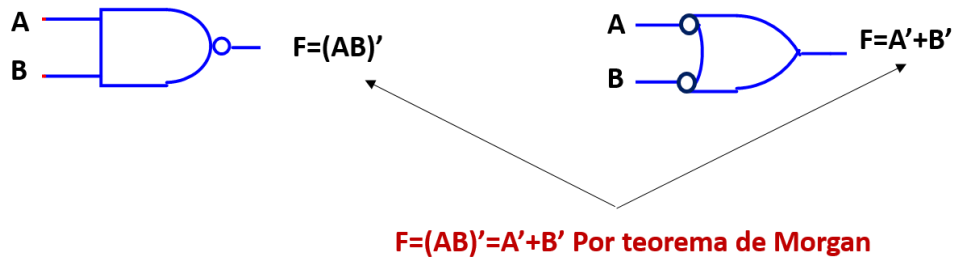


Observe que las compuertas G_2 y G_3 actúan como una compuerta AND y G_1 como una compuerta OR. Además puesto que un círculo indica una inversión, dos círculos conectados representan una doble inversión y, por tanto, se cancelan entre sí.

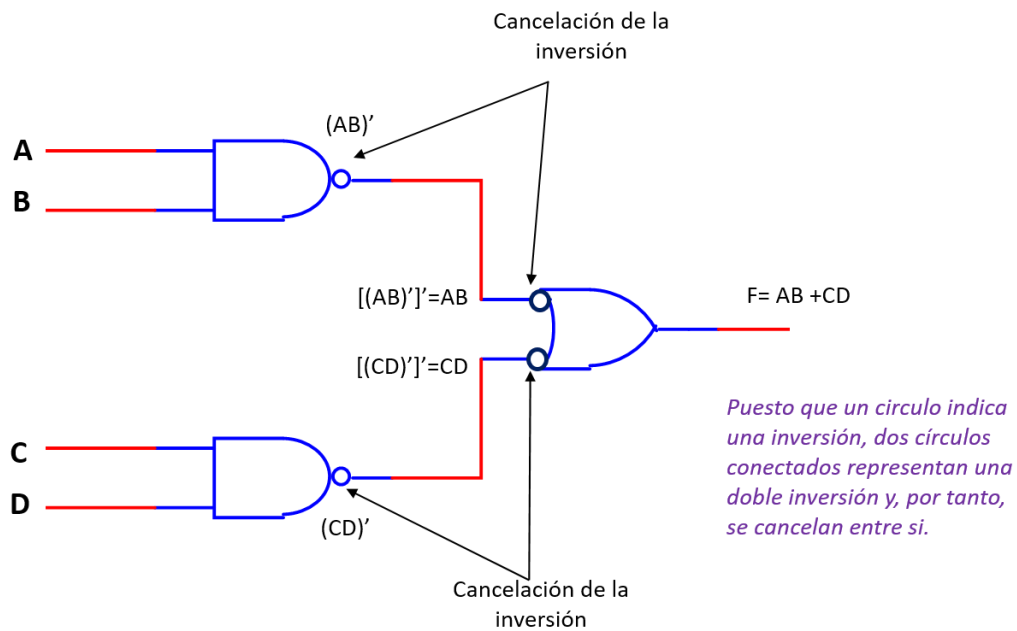
Diagrama Lógico usando compuertas lógicas duales.

Todos los diagramas lógicos que utilizan compuertas NAND debería dibujarse utilizando el símbolo NAND o el símbolo equivalente Negativa-OR para representar cada compuerta, con el fin de reflejar la operación de la compuerta dentro del circuito lógico. Los símbolos NAND y Negativa-OR se denominan símbolos duales. Cuando se dibuja un diagrama lógico NAND, siempre se emplean los símbolos de compuerta de tal forma que cada una de las conexiones entre la salida de una compuerta y la entrada de otra sea una conexión círculo-círculo o una conexión no círculo-no círculo. En la figura se puede observar que estas compuertas son equivalentes.

Práctica #4 La propiedad universal de las compuertas NAND y NOR.



Usar compuertas duales facilita el análisis y la conversión a compuertas universales. En la figura se observa la función F_1 que se ha implementado pero usando compuertas Negativa-OR.



Práctica #4 La propiedad universal de las compuertas NAND y NOR.**La compuerta NOR como elemento lógico universal**

La función NOR es la dual de la función NAND. Por esta razón, todos los procedimientos y las reglas para la lógica NOR son el dual de los procedimientos y reglas correspondientes que se desarrollaron para la lógica NAND. En la figura se muestra la equivalencia en NOR de las compuertas básicas.

COMPUERTA BÁSICA	EQUIVALENCIA NOR

Lógica NOR

La implementación de una función booleana con compuertas NOR requiere que la función se simplifique en la forma de *producto de sumas*. Una expresión en producto de sumas especifica un conjunto de compuertas OR para los términos suma, seguidos por una compuerta AND para obtener el producto.

Por ejemplo:

La función F_2 no es un producto de sumas por lo que hay que simplificar la función.

$$F_2 = [(A+B)' + (C+D)']'$$

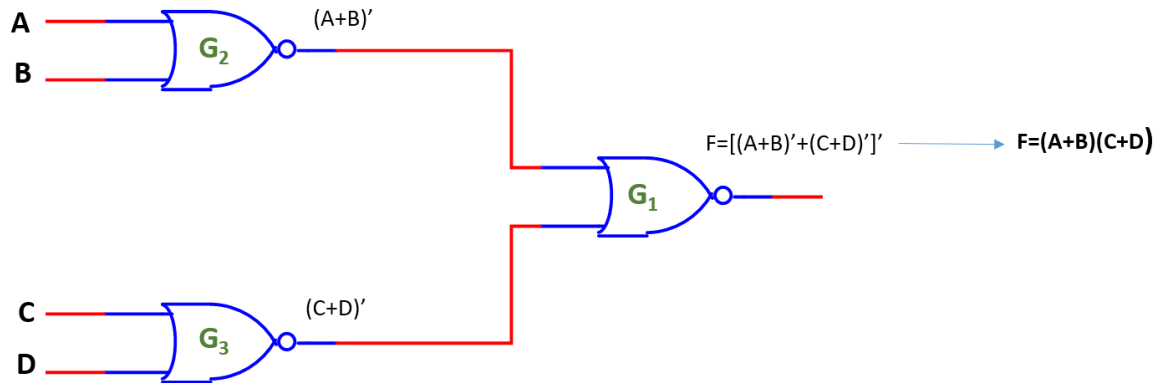
$$F_2 = [A'B' + C'D']$$

$$F_2 = (A'B')'(C'D')'$$

$$F_2 = (A+B)(C+D)$$

Práctica #4 La propiedad universal de las compuertas NAND y NOR.

En la siguiente figura se muestra la función implementada con compuertas NOR.



Observe que las compuertas G_2 y G_3 actúan como una compuerta OR y G_1 como una compuerta AND. Además puesto que un círculo indica una inversión, dos círculos conectados representan una doble inversión y, por tanto, se cancelan entre sí.

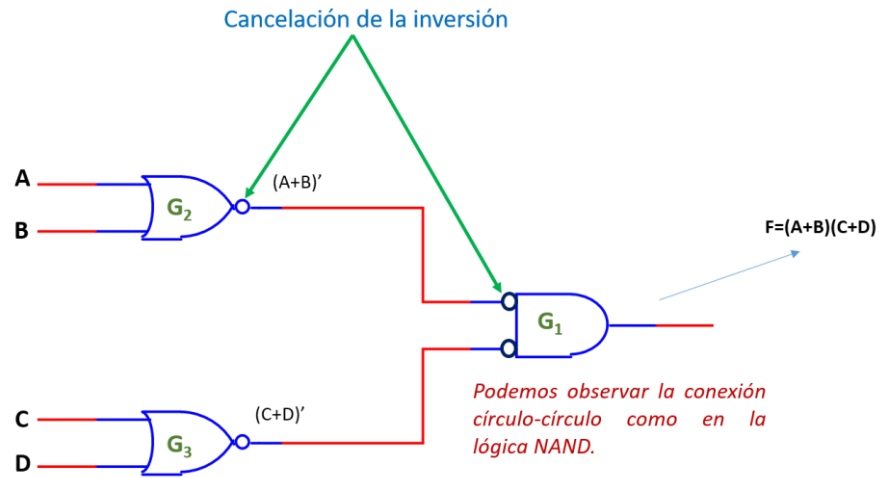
Diagrama Lógico usando compuertas lógicas duales.

El dual de la compuerta NOR es la compuerta Negativa-AND y de igual manera usar compuertas duales facilita el análisis. En la figura se muestra la dualidad de estas compuertas.



Práctica #4 La propiedad universal de las compuertas NAND y NOR.

En la figura se observa la función F_2 que se ha implementado pero usando compuertas Negativa-AND.



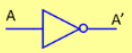
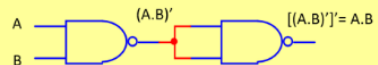
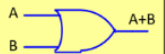
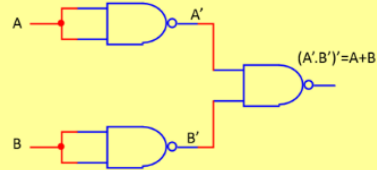
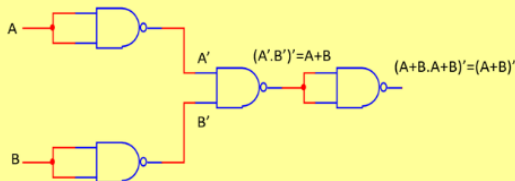
Práctica #4 La propiedad universal de las compuertas NAND y NOR.**Desarrollo de la práctica:**

Materiales:

- 1 Display de 7 segmentos de cátodo común.
- Compuertas NAND 74LS00
- Compuertas NOR 74LS02
- 1 DIPSWITCH DE 4
- 1 RESISTENCIAS DE 470 Ω
- 4 Resistencias de 220 Ω o 330 Ω
- 1 Arduino (Opcional).

Práctica 5.1 Comprobando la universalidad de la compuerta NAND y NOR.

Instrucciones: Comprueba la equivalencia de cada una de las compuertas básicas implementando con compuertas 74LS00 y comprueba la tabla de la verdad.

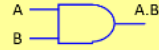
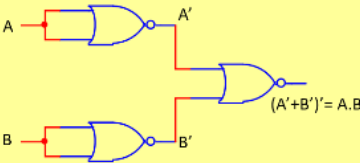
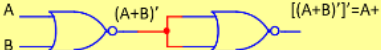
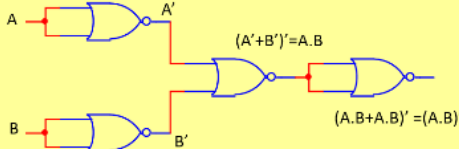
COMPUERTA BÁSICA	EQUIVALENCIA NAND
	 $(A.A)' = A'$
	 $[(A.B)']' = A.B$
	 $(A'.B')' = A+B$
	 $(A'.B')' = A+B$ $(A+B.A+B)' = (A+B)'$

Reporta:

Fotos de los circuitos implementados.

Práctica #4 La propiedad universal de las compuertas NAND y NOR.

Instrucciones: Comprueba la equivalencia de cada una de las compuertas básicas implementando con compuertas 74LS02 y comprueba la tabla de la verdad.

COMPUERTA BÁSICA	EQUIVALENCIA NOR
 $A \rightarrow A'$	 $(A+A)' = A'$
 $A, B \rightarrow A.B$	 $(A'+B')' = A.B$
 $A, B \rightarrow A+B$	 $[(A+B)']' = A+B$
 $A, B \rightarrow (A.B)'$	 $(A'+B')' = A.B$ $(A.B+A.B)' = (A.B)'$

Reporta:

Fotos de los circuitos implementados.

Práctica 5.2 Implementación de funciones usando compuertas NAND y NOR.

Instrucciones: Implementa la siguiente función $F = X'Y'Z' + XYZ'$ con compuertas NAND y con compuertas NOR.

NOTA: No olvides que hay que llevar la función a suma de productos o producto de sumas si es necesario.

Reporta:

1. Pasos de la conversión de suma de productos o producto de sumas si es necesario.
2. Diagrama del circuito con compuertas NAND y NOR. Puedes usar símbolos duales.
3. Fotos del circuito.

Conclusiones Individuales

Integrantes: _____

Objetivo: El alumno simulará circuitos combinacionales, en particular sumadores y restadores usando el software Multisim.

MARCO TEÓRICO

Lógica Combinacional

Un **Circuito Combinacional** consiste en arreglos de compuertas lógicas cuyas salidas en cualquier momento están determinadas por la combinación actual de entradas

Un circuito combinacional realiza una operación que se puede especificar lógicamente con un conjunto de funciones booleanas. La estructura general de un circuito combinacional se muestra en la *figura 6.1*.



Figura 6. 1 Estructura general de un circuito combinacional.

Procedimiento de diseño de un circuito combinacional

El diseño de los circuitos combinacionales surge del planteamiento verbal del problema y termina en un diagrama lógico, o un conjunto de funciones booleanas del cual puede obtenerse con facilidad el diagrama lógico.

El procedimiento sigue los siguientes pasos:

1. Se enuncia el problema.
2. Se determina el número de variables de entrada disponibles y de las variables de salida requeridas.
3. Se asignan símbolos de letra a las variables de entrada y salida.

Práctica V. Simulación de circuitos combinacionales: Sumadores y Restadores

4. Se deriva la tabla de verdad que define las relaciones requeridas entre las entradas y salidas.
5. Se obtiene la función booleana simplificada para cada salida.
6. Se dibuja el diagrama lógico.

En cualquier aplicación particular ciertas restricciones, limitaciones y criterios servirán como guía en el proceso de escoger una expresión algebraica particular. Un método práctico de diseño sería tener que considerar tales restricciones como:

1. Número mínimo de compuertas.
2. Numero mínimo de entradas a una compuerta.
3. Tiempo mínimo de propagación de la señal a través del circuito.
4. Número mínimo de interconexiones.
5. Limitaciones de las capacidades de impulsión de cada compuerta.

DESARROLLO DE LA PRÁCTICA

Equipo y software necesario:

- 1 Laptop
- Simulador Multisim de National Instruments.

Práctica 6.1 Implementación de un medio sumador.

Instrucciones: *Lee los siguientes pasos para el diseño de un medio sumador e implementa el circuito en el simulador y comprueba la tabla de verdad.*

Definición: Un circuito combinacional que lleva a cabo la adición de dos bits se denomina **medio sumador**.

Procedimiento de diseño:

1. **Se enuncia el problema:** “Implementar un medio sumador.”
2. **Se determina el número de variables de entrada disponibles y de las variables de salida requeridas.** Un medio sumador requiere de dos entradas ya que este solamente suma dos bits. Por otro lado requiere de dos salidas, una para el resultado de la suma y otra para el acarreo.

Práctica V. Simulación de circuitos combinacionales: Sumadores y Restadores

3. **Se asignan símbolos de letra a las variables de entrada y salida:** Las entradas se llamarán **X** y **Y**. A las salidas se designarán las letras **S** para la suma y **C** para el acarreo.
4. **Se deriva la tabla de verdad 6.1 que define las relaciones requeridas entre las entradas y salidas.** La tabla de la verdad se puede observar en la *figura 6.2*.

Tabla 6.1 Tabla de la verdad de un medio sumador.

X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

5. **Se obtiene la función booleana simplificada para cada salida.**

$$S = X'Y + XY'$$

$$C = XY$$

6. **Se dibuja el diagrama lógico del medio sumador, figura 6.3.**

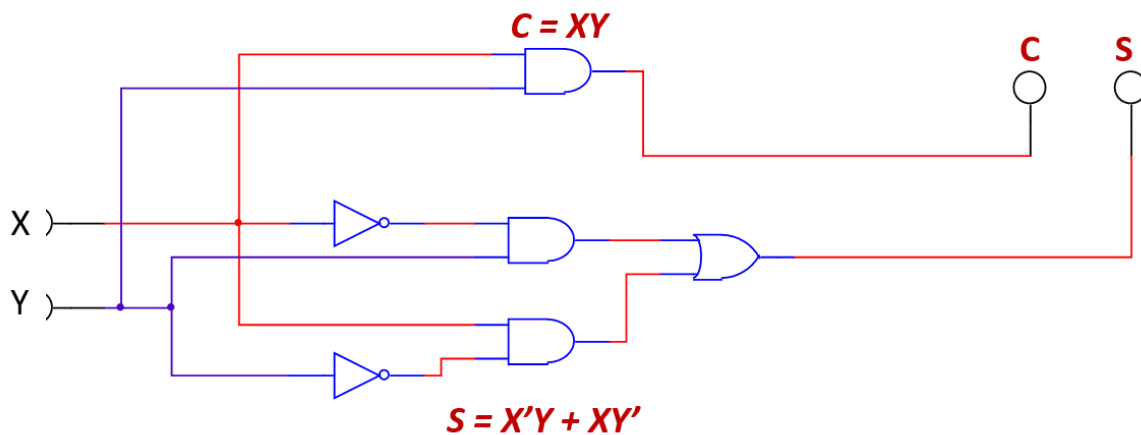


Figura 6. 2 Circuito de un medio sumador.

Práctica 6.2 Implementación de un sumador completo.

Instrucciones: *Implementa un sumador completo en Multisim.*

Definición: Un **sumador completo** es un circuito combinacional que realiza la suma aritmética de tres bits. Consta de tres entradas y dos salidas

Para el diseño de un sumador completo se siguen los procedimientos anteriores y se obtiene la *tabla 6.2*.

Tabla 6.2 Tabla de la verdad de un sumador completo.

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Las funciones simplificadas se pueden obtener usando mapas “K”.

		YZ			
		00	01	11	10
X	0	0	0	1	0
	1	0	1	1	1

MAPA DE KARRNAUGHT PARA C

$$C = XY + XZ + YZ$$

		YZ			
		00	01	11	10
X	0	0	1	0	1
	1	1	0	1	0

MAPA DE KARRNAUGHT PARA S

$$S = X'Y'Z + X'YZ' + XY'Z' + XYZ$$

Se obtiene el diagrama con compuertas a partir de las funciones booleanas.

Ejercicio: Implementa el circuito de la *figura 6.3* en el simulador y comprueba la tabla de la verdad.

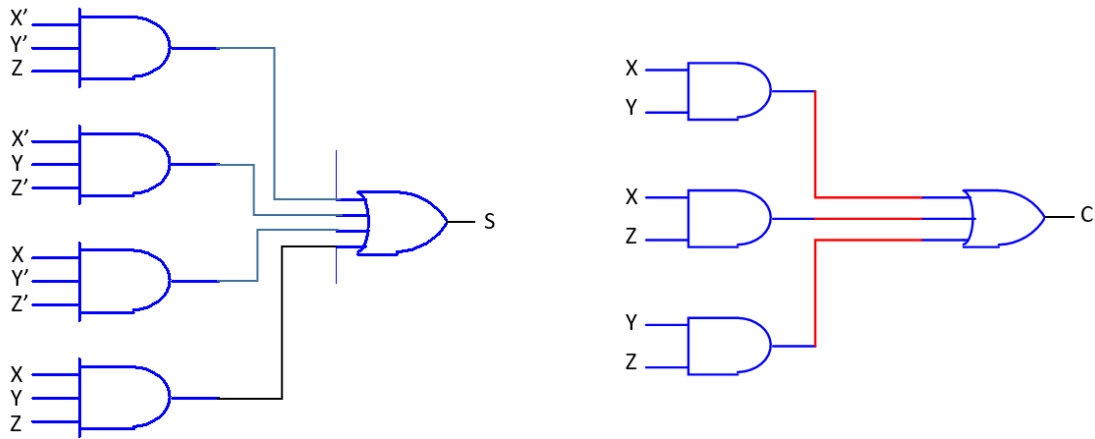


Figura 6. 3 Circuito combinacional de un sumador completo.

Práctica 6.3 Implementación de un sumador de 5 bits.

Hasta ahora se ha visto el medio sumador y el sumador completo, este último sólo puede sumar hasta tres bits como se vio anteriormente. Entonces para sumar cifras binarias más grandes se requiere hacer arreglos de sumadores, como se muestra en la siguiente *figura 6.4*.

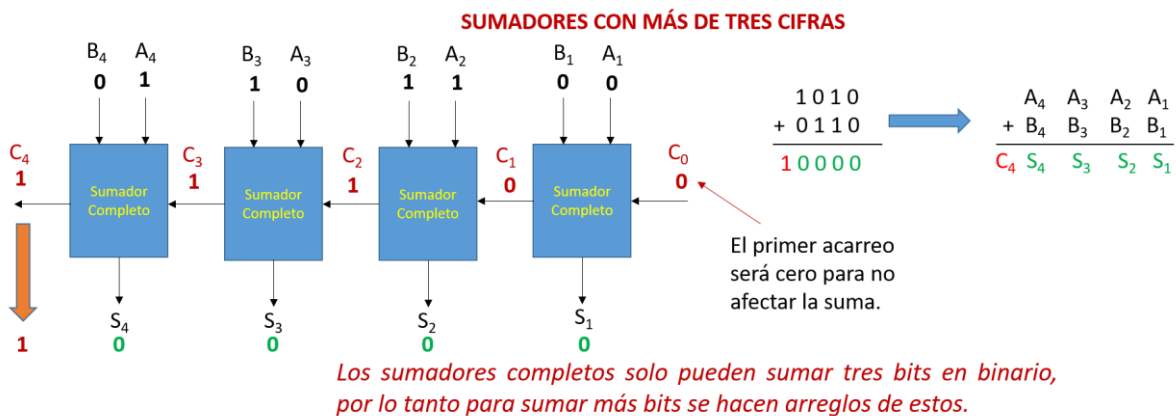


Figura 6. 4 Sumador de 4 bits.

Práctica V. Simulación de circuitos combinacionales: Sumadores y Restadores

Observe que el bit de acarreo “C” se lleva hacia el siguiente sumador, sumado el acarreo a los dos bits establecidos denotados con las letras **A** y **B**. Los sumadores se conectan en serie para poder realizar la suma.

Instrucciones: *Implementa un sumador de 5 bits interconectando 5 sumadores completos. Para la conexión puedes basarte de la figura 6.4.*

En el menú de dispositivos TTL del simulador, localiza el integrado “**74LS183**”. Este integrado es un sumador completo y se puede observar en la figura 6.5. Recuerda que deberás conectarlos en serie.

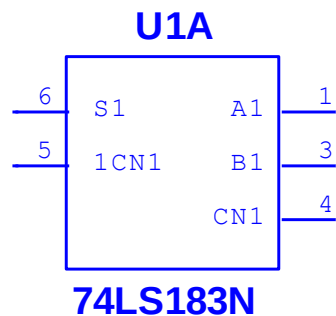


Figura 6. 5. Circuito integrado 74LS183N: sumador completo.

Las terminales 1, 3 y 4 corresponden a las entradas del sumador. La entrada **CN1** corresponde a la terminal donde deberás ingresar el acarreo del sumador anterior. Recuerda que la terminal **CN1** del primer sumador deberá ir a tierra para no afectar la suma. Por otro lado está la terminal **S1** que es el resultado de la suma y la terminal **1CN1** es el acarreo de salida que deberá ir al siguiente sumador.

Tanto las entradas como las salidas deben de tener indicadores ya sean LEDs o PROBEs, figura 6.6, para visualizar los bits que ingresan y de igual manera para el resultado de la suma.

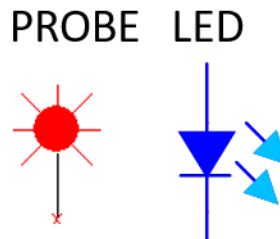


Figura 6. 6 Indicadores.

Usa una conexión PULL-DOWN para ingresar los bits de la suma con interruptores. Una vez armado el circuito puedes iniciar comprobando la siguiente operación.

$$\begin{array}{r}
 10110 \quad 22 \\
 + 11101 \quad 29 \\
 \hline
 110011 \quad 51
 \end{array}$$

Si todo salió correcto comprueba otras operaciones de suma.

Práctica 6.4. Implementación de un medio restador.

Instrucciones: *Implementa con compuertas lógicas un medio restador en multisim, usando las funciones obtenidas de la tabla de la verdad.*

Definición: Un **medio restador** es un circuito combinacional que realiza la operación de resta con dos bits, en la *tabla 6.3* se puede observar el funcionamiento de este circuito.

Tabla 6.3 Tabla de la verdad de un medio restador.

X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

$$D = X'Y + XY'$$

$$B = X'Y$$

El medio restador tiene dos entrada X, Y y dos salidas **B**, **D**. La salida B es el acarreo prestado (Borrow) y D la diferencia resultante de la resta.

Práctica 6.5 Implementación de un restador completo.

Instrucciones: *Implementa con compuertas un restador completo en multisim, usando las funciones obtenidas de la tabla de la verdad.*

Definición: Un **restador completo** es un circuito combinacional que lleva a cabo una sustracción entre tres bits, tomando en cuenta que un 1 se ha tomado por una etapa significativa más baja. Este circuito tiene tres entradas y dos salidas. Las tres entradas X, Y y Z, denotan al minuendo, sustraendo y a la toma previa, respectivamente. Las dos salidas, D y B, representan la diferencia y la salida tomada, respectivamente. La *tabla 6.4* describe el funcionamiento de este circuito.

Tabla 6.4 Tabla de la verdad de un restador completo.

X	Y	Z	B	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D = X'Y'Z + X'YZ' + XY'Z' + XYZ$$

$$B = X'Y + X'Z + YZ$$

Prácticas 6.6 Implementación de un restador de 4 bits.

Hasta ahora se ha visto el medio restador y el restador completo, este último sólo puede restar hasta tres bits como se vio anteriormente. Entonces para realizar operaciones con cifras binarias más grandes se requiere hacer arreglos de estos, como se muestra en la *figura 6.7*.

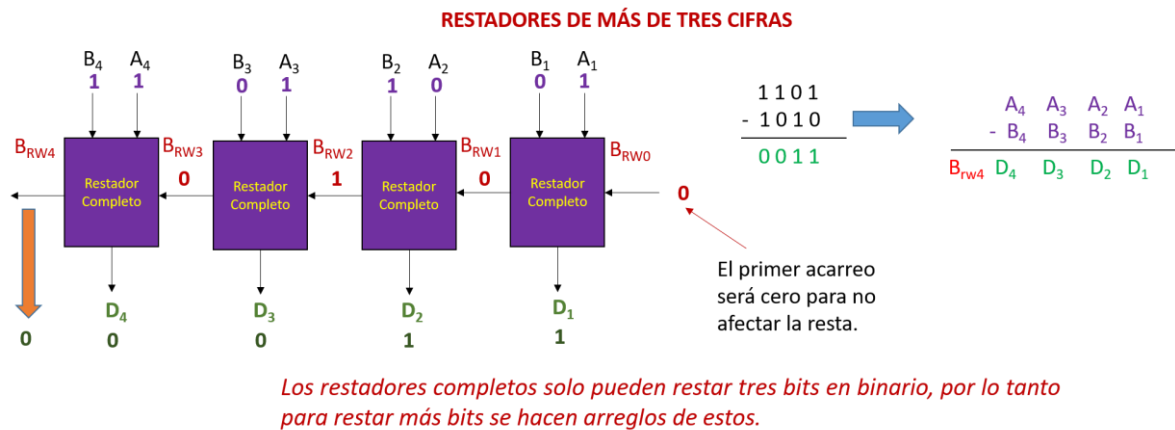


Figura 6. 7 Restador de 4 Bits.

Observe que el bit de acarreo “**B_{RW}**” se lleva hacia el siguiente restador; este es el acarreo prestado. Este bit prestado se resta con los bits respectivos **A** y **B**. Es muy importante que el bit **B_{RW}** del primer restador se conecte a cero para no afectar la resta.

Instrucciones: *Implementa un restador de 4 bits basándose en la conexión que se mostró en la figura 6.7. Usa los circuitos obtenidos del restador completo de la práctica 6.5 y conviértelos en un **Subcircuito** como se observa en la figura 6.8.*

NOTA: Si tienes dudas como crear un subcircuito en Multisim, asesórate con tu profesor.

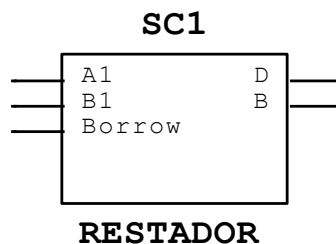


Figura 6. 8. Subcircuito de un restador completo en Multisim.

Reporta el circuito realizado y comprueba su funcionamiento realizando algunas operaciones, puedes usar la siguiente resta como ejemplo para la prueba del circuito.

$$\begin{array}{r}
 1101 \quad 13 \\
 - 1010 \quad 10 \\
 \hline
 0011 \quad 3
 \end{array}$$

En los restadores el *minuendo* debe ser mayor al *sustraendo*. En el ejemplo la resta es **13-10= 3**, sin embargo si fuese al revés la operación **10-13** el resultado de la resta da como resultado el número **29**. Esto es porque estamos ante la presencia de un número negativo por lo que hay que sacar su **complemento A2** del resultado, como se observa en la *figura 6.9*. Para estos casos se tiene que implementar un circuito adicional que realice la operación del complemento **A2**.

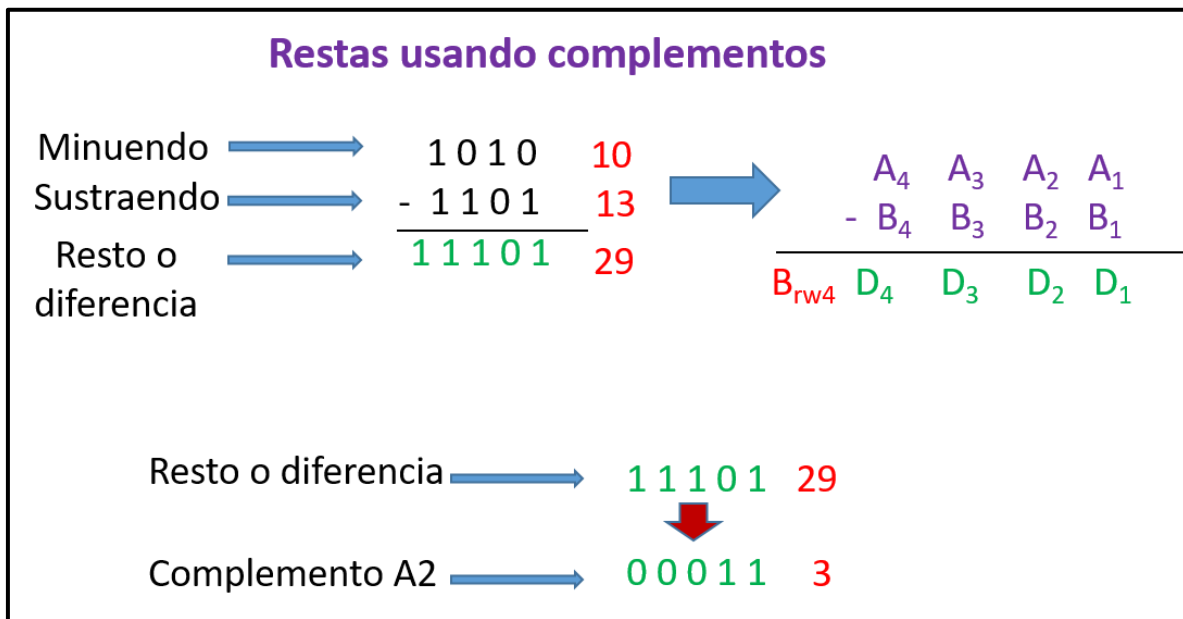


Figura 6. 9. Complemento A2 para operaciones de resta.

La sustracción de dos números binarios puede llevarse a cabo tomando el complemento del sustraendo y agregándoselo al minuendo. Por este método, la operación de sustracción llega a ser una operación que requiere sumadores completos para su implementación en máquina.

Conclusiones Individuales

Integrantes: _____

Objetivo: El alumno implementará circuitos combinacionales (codificadores, decodificadores, multiplexores y demultiplexores) usando circuitos integrados en la categoría MSI (Integración a media escala).

MARCO TEÓRICO

Introducción

Los sistemas digitales obtienen datos codificados en binario e información con la que operan en cierta forma continua. Algunas de las operaciones son:

1. Decodificación y codificación.
2. Multiplexaje.
3. Demultiplexaje.
4. Comparación.
5. Conversión de Código.
6. Buses de datos.

Todas las operaciones mencionadas anteriormente y otras más se han facilitado gracias a la disponibilidad de numerosos CIs (Circuitos Integrados) en la categoría MSI (Integración a mediana Escala). Si se habla a nivel componente los CIs del tipo MSI tienen entre 100 a 1000 componentes integrados en un encapsulado. Si se habla de número de compuertas lógicas, esos componentes equivalen de 12 a 100 compuertas dentro del encapsulado. Hay que recordar que las compuertas lógicas están fabricadas por transistores que pueden ser del tipo BJT o MOSFET por mencionar un ejemplo. En la tabla 7.1 Se pueden ver los diferentes niveles de integración.

Tabla 7.1 Niveles de Integración de los CIs.

Nivel de integración	No. De componentes	No. De compuertas
Pequeña escala de integración (SSI)	10 a 100	1 a 12
Mediana Escala de Integración (MSI)	100 a 1 000	12 a 100
Gran Escala de Integración (LSI)	1 000 a 10 000	100 a 1 000
Muy gran Escala de Integración (VLSI)	10, 000 A 100 000	1 000 A 10 000
Ultra gran escala de Integración (ULSI)	100 000 a 1 000 000	10 000 a 100 000
Giga gran escala de Integración (GLSI)	Más de 1 000 000	Más de 100 000

Codificadores y decodificadores

Un **codificador** es un circuito lógico combinacional que, esencialmente, realiza la función “inversa” del **decodificador**. Un codificador permite que se introduzca en una de sus entradas un nivel activo que representa un dígito, como puede ser un dígito decimal u octal y lo convierte a una salida codificada, como BCD o binario. El proceso de conversión de símbolos comunes o números a un formato codificado recibe el nombre de codificación.

Ejemplos de codificadores:

- Codificador Decimal-BCD.
- Codificador de 8-lineas a 3-lineas.

Ejemplos de decodificadores:

- Decodificador de Binario a Decimal.
- Decodificador de BCD a 7 segmentos.

Multiplexores

Un **multiplexor (MUX)** es un dispositivo que permite dirigir la información digital procedente de diversas fuentes a una única línea para ser transmitida a través de dicha línea a un destino común. El multiplexor básico posee varias líneas de entrada de datos y una única línea de salida. También posee entradas de selección de datos, que permite conmutar los datos digitales provenientes de cualquier entrada hacia la línea de salida. A los multiplexores también se les conoce como selectores de datos.

Demultiplexores

Un **demultiplexor (DEMUX)** básicamente realiza la función contraria a la del multiplexor. Toma datos de una línea y los distribuye a un determinado número de líneas de salida. Por este motivo el demultiplexor se conoce también como distribuidor de datos.

DESARROLLO DE LA PRÁCTICA

MATERIAL

- codificador de decimal a binario 74LS147
- 1 Decodificador de BCD a 7 segmentos 74LS47
- 1 Multiplexor 74LS151
- 13 Resistencias de 330 Ω
- 4 LED's del mismo color de preferencia.
- 1 DIPSWITCH de 9 interruptores
- 1 Display de cátodo común.
- 1 Protoboard
- Cables de conexión

EQUIPO

- 1 Generador de funciones.

Práctica 7.1 Codificador de Decimal a Binario.

El **74LS147** de la *figura 7.0* es un codificador de decimal a binario. Sus entradas y salidas están negadas. El integrado se alimenta con 5V en su terminal 16. La terminal 8 se conecta a tierra.

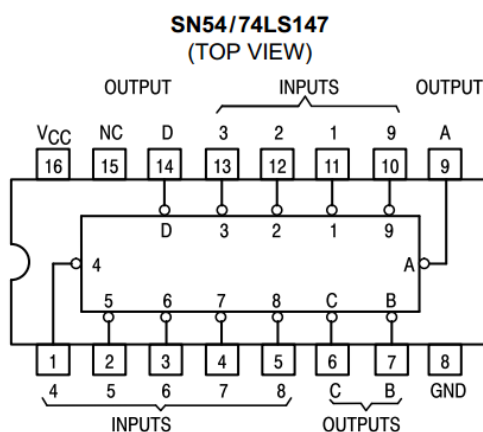


Figura 7. 1 Codificador de Decimal a Binario 74LS147.

El circuito equivalente en compuertas lógicas dentro del encapsulado se puede ver en la *figura 7.2*. Puede observarse el nivel de Integración del tipo MSI.

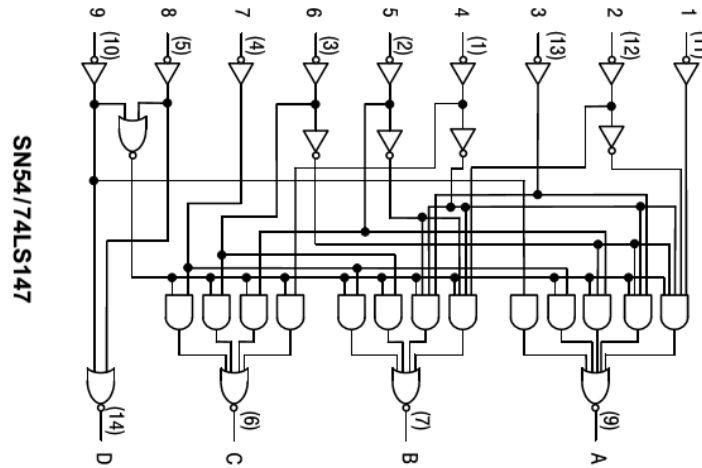


Figura 7. 2. Circuito lógico dentro de un CI 74LS147.

Instrucciones: Implementa el circuito de la figura 7.3 en protoboard. Se trata de un codificador de decimal a binario usando el integrado 74LS147.

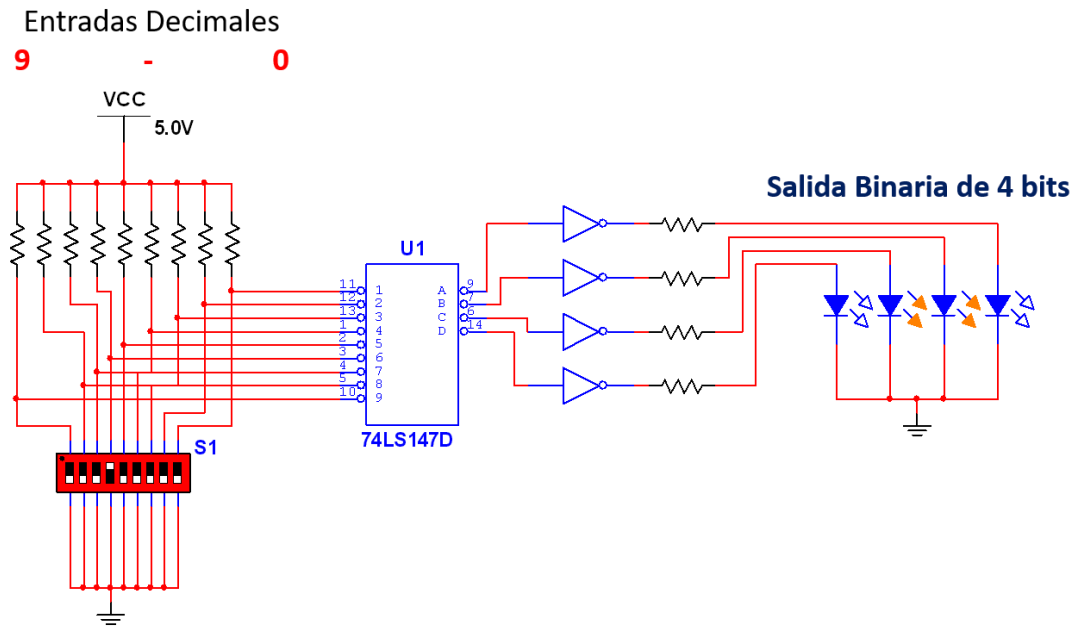


Figura 7. 3. Implementación del codificador Decimal a Binario.

Cada uno de los interruptores representan un dígito decimal, de derecha a izquierda van del 1 al 9. Dependiendo del interruptor encendido, el codificador debe de mostrar a la salida su equivalente en binario. Si todos los interruptores están abiertos se entiende que es el número cero decimal y a la salida no debe encender ningún LED. En la *figura 7.3* se muestra que el interruptor número 6 de la entrada está encendido y a la salida se muestra el 6 (0110) en binario, siendo el LED de la derecha el menos significativo.

En la *figura 7.3* también se puede observar que la configuración del interruptor tipo DipSwitch (**S1**) está en conexión PULL-UP, esto es debido a que las entradas del 74LS147 se encuentran negadas, por lo tanto cuando el interruptor este en ON la salida realmente será un cero lógico y la entrada lo negará dando un estado alto. De igual manera la salida del codificador esta negada por lo que hay que usar compuertas NOT para tener la salida deseada. Esto es por si se desea hacer el análisis con estados altos (HIGH).

Práctica 7.2 Decodificador BCD a 7 segmentos

El 74LS47 de la *figura 7.4* es un decodificador el cual permite ingresar combinaciones de 1 y 0 en código BCD y traducirlas a 7 segmentos. Sus salidas están negadas por lo que está fabricado para trabajar con un display de ánodo común, sin embargo se puede usar compuertas NOT a la salida para usar un display de cátodo común. El integrado se alimenta con 5V en su terminal 16. La terminal 8 se conecta a tierra. El circuito equivalente en compuertas lógicas dentro del encapsulado se puede ver en la *figura 7.5*.

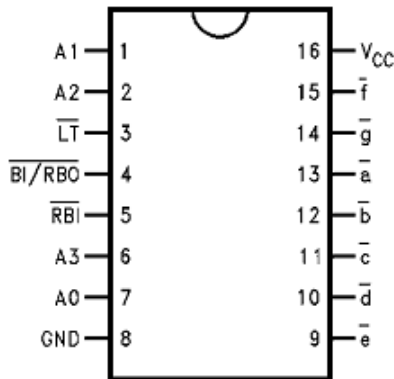
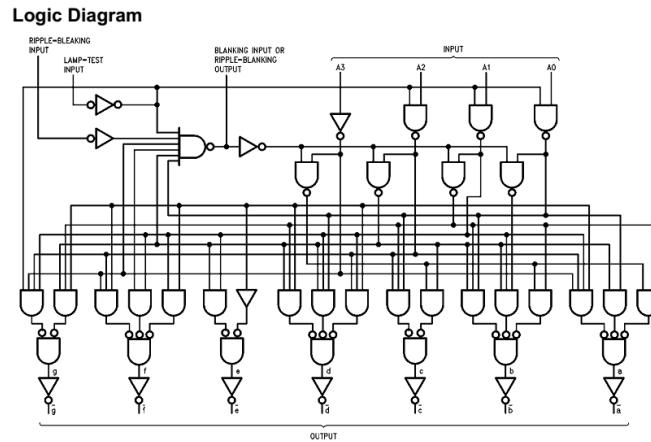


Figura 7. 4. Codificador de BCD a 7 segmentos 74LS47.



Numerical Designations—Resultant Displays

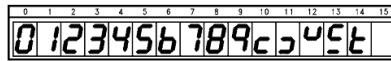


Figura 7. 5. Circuito lógico dentro de un CI 74LS47.

Instrucciones: Implementa el circuito de la figura 7.6 en protoboard. Se trata de un decodificador de BCD a 7 segmentos usando el integrado 74LS47.

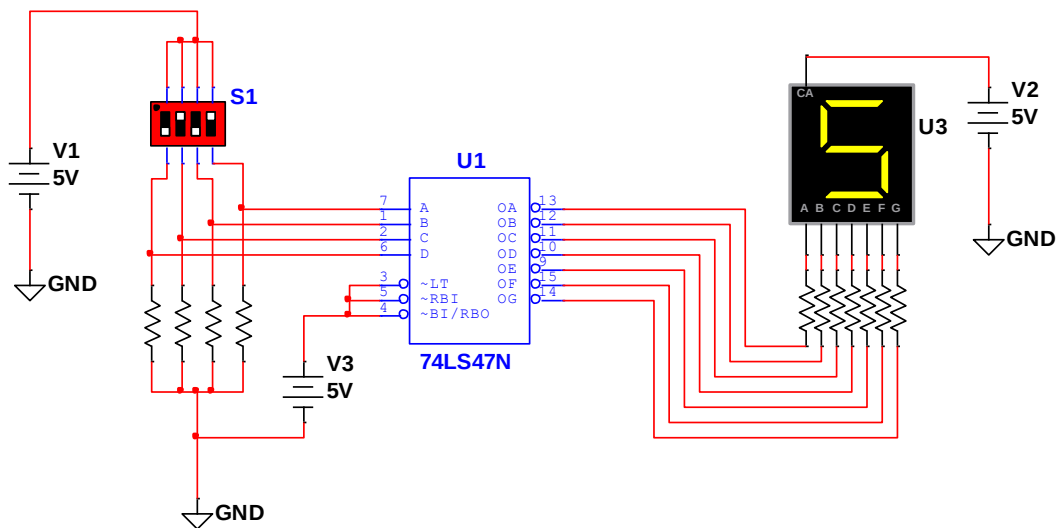


Figura 7. 6. Implementación del decodificador de BCD a 7 segmentos con display de ánodo común.

El display es de ánodo común ya que las salidas están negadas, si se desea usar uno de cátodo común use compuertas NOT para negar las salidas. Las resistencias que van a cada uno de los segmentos pueden ser eliminadas y poner un resistencia de $470\ \Omega$ en el ánodo común del display.

Las entradas binarias son **A-D**, las salidas a 7 segmentos son de **a-g**.

La entrada **LT** (Lamp Test Input) se usa para verificar que ninguno de los segmentos este fundido. Para realizar la comprobación se debe mandar **LT** en pulso bajo y la entrada **BI/RBO** en nivel alto para que se enciendan todos los segmentos del display. Como no se usará esta función mande las terminales 3,4 y 5 a un pulso alto.

Si se desea usar un display de cátodo común use el circuito de la *figura 7.7*.

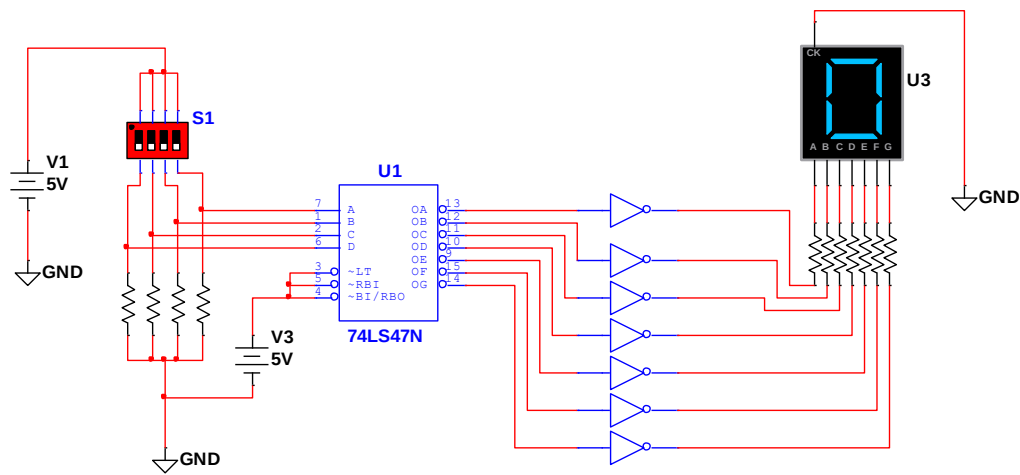


Figura 7. 7. Implementación del decodificador de BCD a 7 segmentos con display de cátodo común.

En la figura 7.9 se muestra el integrado 74LS151. La terminal 16 se alimenta con 5 V y su conexión a tierra en la terminal 8.

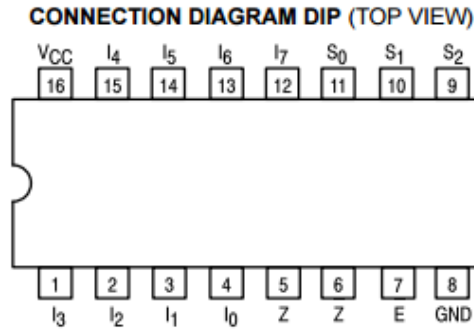


Figura 7. 9. Multiplexor 74LS151.

El multiplexor tiene las siguientes compuertas en su encapsulado, *figura 7.10*.

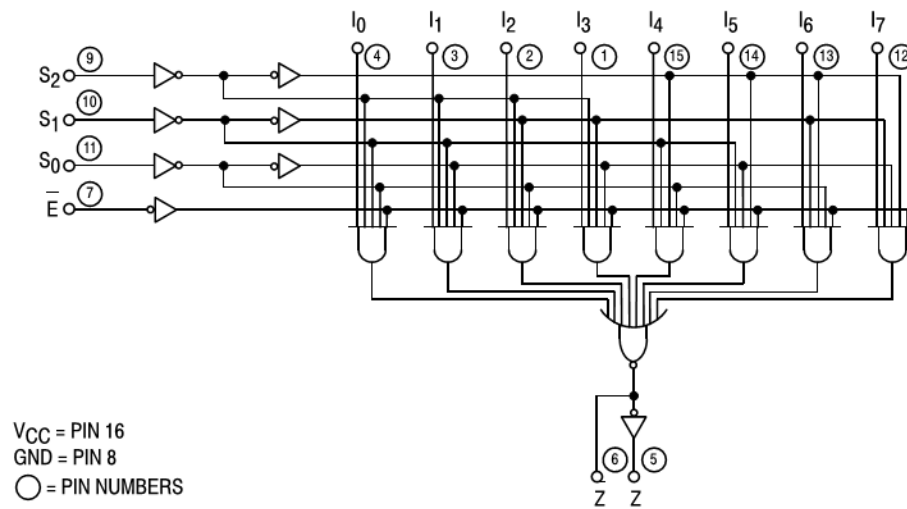


Figura 7. 10. Circuito lógico dentro de un CI 74LS151.

Instrucciones: Implementa el circuito de la figura 7.11 en protoboard. Se trata de un multiplexor usando el integrado 74LS151.

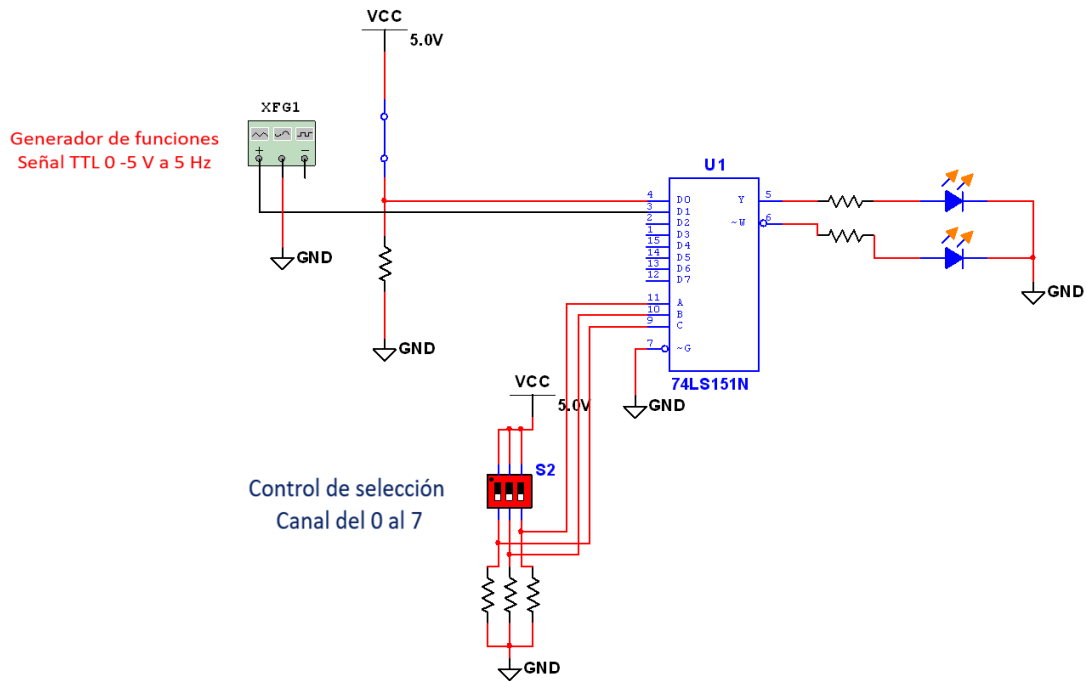


Figura 7. 11. Implementación del Multiplexor 74LS151.

En este circuito solo se usarán dos canales de información **D0** y **D1**. La entrada D0 tiene un valor fijo de 0 o 1, mientras que en la entrada D1 se ingresará una señal TTL que hará parpadear los LED's de salida.

Conclusiones Individuales

Integrantes: _____

Objetivo: El alumno implementará circuitos secuenciales “Flip-Flops”, que le ayudarán a entender cómo funcionan los elementos de memoria en circuitos digitales.

MARCO TEÓRICO

Introducción

Los circuitos digitales que hasta ahora se han considerado han sido *combinacionales*, esto es, las salidas en cualquier momento dependen por completo de las entradas presentes en el tiempo.

Por otro lado los **Circuitos Secuenciales** usan elementos de almacenamiento además de compuertas lógicas, y sus salidas son función de las entradas y del estado de los elementos de almacenamiento. Esto último, a su vez, es función de entradas anteriores.

Los dispositivos *biestables* se dividen en dos categorías:

- Latches.
- Flip-flops

Los biestables son aquellos circuitos que poseen dos estados estables, denominados *SET (activación)* y *RESET (desactivación)*, en los cuales se pueden mantener información indefinidamente, lo que le hace muy útiles como dispositivos de almacenamiento.

Dependiendo de sus entradas los biestables se dividen:

- **Asíncronos:** Sólo tienen entradas de control. El más empleado es el biestable RS.
- **Síncronos:** Además de las entradas de control posee una entrada de sincronismo o de reloj.

La diferencia básica entre latches y flip-flops es la manera que cambian de un estado a otro. Los flip-flops son bloques básicos de construcción de los contadores, registros y otros circuitos de control secuencia, y se emplean también en ciertos tipos de memorias.

El latch S-R (SET-RESET)

Un latch (late memory en inglés) es un circuito electrónico biestable asíncrono usado para almacenar información en sistemas lógicos digitales. Un latch puede almacenar un bit de información, asimismo los latches se pueden agrupar de tal manera que logren almacenar más de 1 bit, por ejemplo el 'latch quad ' (capaz de almacenar cuatro bits) y el 'latch octal' (capaz de almacenar ocho bits).

Los latches son dispositivos biestables asíncronos que no tienen entrada de reloj y cuyo cambio en los estados de salida es función del estado presente en las entradas y de los estados previos en las salidas (retroalimentación). Los latches a diferencia de los flip-flops no necesitan una señal de reloj para su funcionamiento.

Un Latch SR (SET-RESET) con entrada activa a nivel ALTO se compone de dos compuertas **NOR** acopladas, tal como se muestra en la *figura 8.1*; un Latch S'-R' con entrada activa a nivel BAJO está formado por dos compuertas **NAND** conectadas tal y como se muestra en la *figura 8.2*. Observe que la salida de cada compuerta se conecta a la entrada de la compuerta opuesta. Esto origina la realimentación (feedback) regenerativa característica de todos los Latches y Flip-Flops.

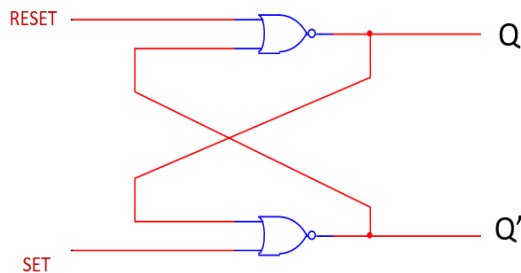


Figura 8. 1 Latch-SR con entrada activa a nivel alto.

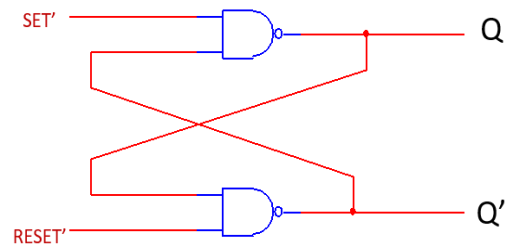


Figura 8. 2 Latch-SR con entrada activa a nivel bajo.

Flip-Flops

Los **biestables (flip-flop en inglés)** son dispositivos síncronos de dos estados, también conocidos como multivibradores biestables.

En este caso, el término síncrono significa que la salida cambia de estado únicamente en un instante específico de una entrada de disparo denominada reloj (CLK), la cual recibe el nombre de entrada de control, C. Esto significa que los cambios en la salida se producen sincronizadamente con el reloj.

Un circuito Flip-Flop puede mantener un estado binario en forma indefinida (en tanto se suministre potencia al circuito) hasta que recibe la dirección de una señal de entrada para cambiar de estado.

La diferencia principal entre los diversos tipos de flip-flops está en el número de entradas que poseen y en la manera en la cual las entradas afectan el estado binario.

Un flip-flop disparo por flanco cambia de estado con el flanco positivo (flanco de subida) o con el flanco negativo (flanco de bajada) del impulso de reloj y es sensible a sus entradas sólo en esta transición de reloj.

Se verán cuatro tipos de flip-flops disparados por flanco: **S-R, D, J-K Y T.**

DESARROLLO DE LA PRÁCTICA

MATERIAL

- 2 Compuertas NAND 74LS00
- 2 Compuertas NOR 74LS02
- 1 compuerta AND 74LS08.
- 1 Circuito Integrado LATCH S'R' 74LS279
- 1 Circuito Integrado LATCH tipo D 74LS75.
- 1 Arduino Uno (Opcional)
- 1 Dipswitch o 3 push-buttons
- Cables de conexión.
- 3 LED'S.

EQUIPO Y SOFTWARE.

- 1 Generador de funciones.
- LabView de National Instruments (Opcional).

Práctica 8.1 El latch S-R (SET-RESET)

Definición: Un latch es un tipo de dispositivo lógico biestable o multivibrador. Un Latch SR (SET-RESET) con entrada activa a nivel ALTO se compone de dos compuertas NOR acopladas, su símbolo se muestra en la *figura 8.3*. Este tiene dos salidas, Q y Q', y dos entradas, ajustar (set) y restaurar (reset).

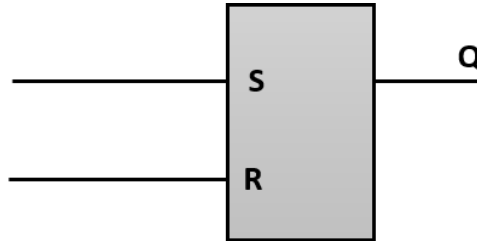


Figura 8. 3 Símbolo de un Latch S-R

Instrucciones: Implementa el Latch S-R de la figura 8.4 con compuertas NOR y comprueba la tabla 8.1, la cual describe el comportamiento del circuito.

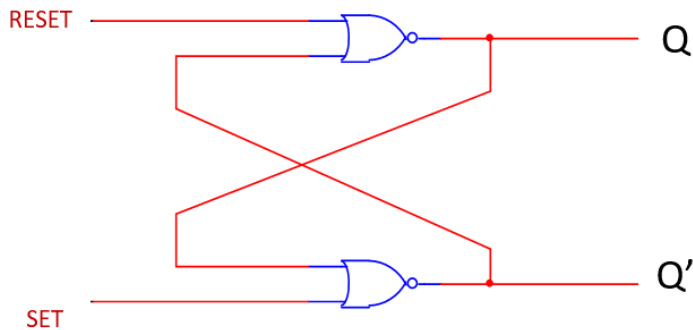


Figura 8. 4 El Diagrama de un Latch SR.

Tabla 8.1 Tabla de la verdad del Latch S-R

FLIP-FLOP SR			
S	R	Q	Q'
1	0	1	0
0	0	1	0
0	1	0	1
0	0	0	1
1	1	0	0

Práctica 8.2 El latch S'-R' (SET-RESET)

Definición: Un Latch S'-R' con entrada activa a nivel BAJO está formado por dos compuertas NAND. Ahora establecer (set) se cambia de posición con resetear (reset). El estado de memoria se da cuando ambas entradas tienen un pulso ALTO. Su símbolo se muestra en la figura 8.5.

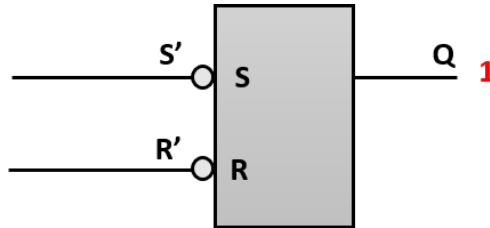


Figura 8. 5 Símbolo de un Latch S'-R'.

Un ejemplo de un Circuito Integrado (CI) en el Latch S'R' 74LS279 que se muestra en la figura 8.6.

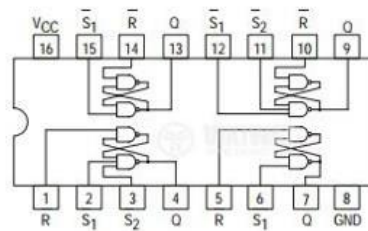


Figura 8. 6 El CI 74LS279

Instrucciones: Implementa el Latch S'-R' de la figura 8-6 con compuertas NAND y comprueba la tabla 8.2, la cual describe el comportamiento del circuito.

Tabla 8.2 Tabla de la verdad del Latch S'-R'

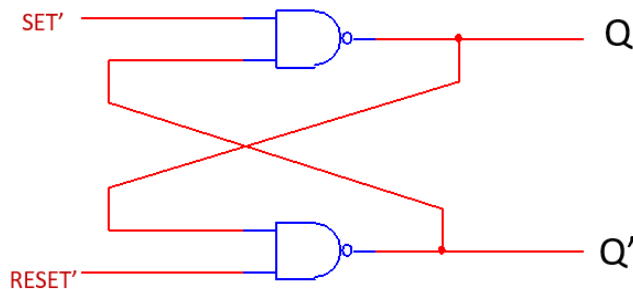


Figura 8. 7 Diagrama del Latch S'-R'.

LATCH S'-R'			
S'	R'	Q	Q'
1	0	0	1
1	1	0	1
0	1	1	0
1	1	1	0
0	0	1	1

Práctica 8.3 El latch tipo D

Definición: El LATCH tipo D se diferencia del LATCH S-R en que sólo tiene una entrada, además de la de habilitación, EN. Esta entrada recibe el nombre de entrada de datos (D). Cuando la entrada D está a nivel ALTO y la entrada EN también, el LATCH se pone en estado SET. Cuando la entrada D está a nivel BAJO y la entrada EN está a nivel ALTO, el LATCH se pone en estado RESET. En otras palabras la salida Q es igual a la entrada D cuando la entrada de habilitación EN este en nivel ALTO. Su símbolo se observa en la *figura 8.8*.

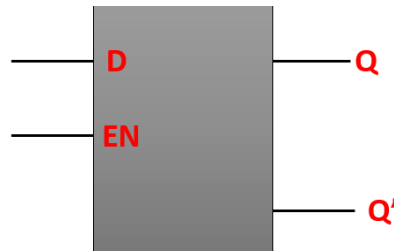


Figura 8. 8 Símbolo del Latch tipo D.

En la figura se puede observar el CI del latch tipo D 74LS75.

Connection Diagram

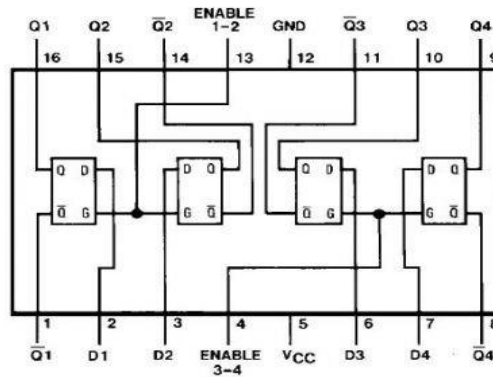


Figura 8. 9 El CI 74LS75.

La X en la tabla representa una condición “indiferente”. En este caso, cuando la entrada EN está a nivel bajo, da lo mismo el valor que tenga la entrada D, ya que las salidas no se ven afectadas y permanecen en los estados en que se encontraban (Memoria).

Instrucciones: Implementa un Latch tipo D de la figura 8.10 con compuertas NAND y comprueba la tabla 8.3, la cual describe el comportamiento del circuito.

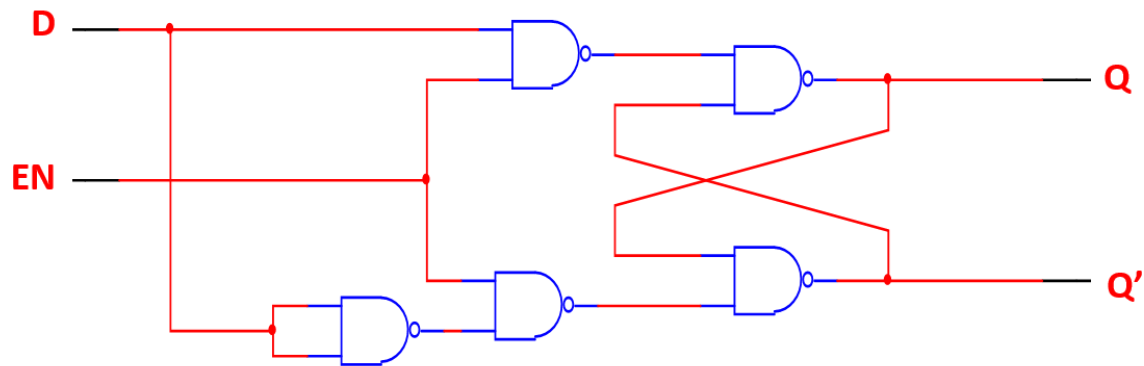


Figura 8. 10 Diagrama del Latch tipo D

Tabla 8.3 Tabla de la verdad del Latch tipo D

D	EN	Q	Q'	Comentarios
0	1	0	1	RESET
1	1	1	0	SET
X	0	Q_0	Q'_0	No hay cambio

Práctica 8.4 Flip-flop RS con disparo de Flanco ascendente.

Definición: El flip-flop RS con reloj consta de un Flip-Flop básico NOR y dos compuertas AND. El Flip-Flop responde a niveles de entrada durante la ocurrencia de un pulso de reloj. Las salidas de las dos compuertas AND permanecen en 0 en tanto que el pulso de reloj (abreviado CLK) sea 0, sin importar los valores de entrada S y R. Cuando el reloj este en pulso bajo no abra cambios. En la figura 8.11 se muestra el símbolo del flip-flop R-S con disparo ascendente.

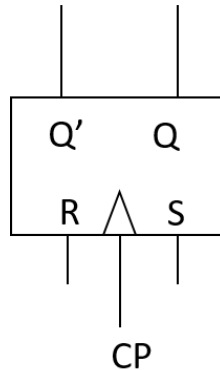


Figura 8. 11 Símbolo de un Flip-Flop R-S con disparo de flanco ascendente.

Cuando el pulso de reloj va en 1, permite que la información de las entradas S-R llegue al FLIP-FLOP básico con compuertas NOR. El estado de ajuste se alcanza con $S=1$, $R=0$ Y $CLK=1$. Para alcanzar el estado despejado, las entradas deben ser $S=0$, $R=1$ Y $CLK=1$. En la tabla 8.4 se muestra el comportamiento del flip-flop R-S con reloj.

Tabla 8.4 Tabla de la verdad simplificada del Flip-Flop R-S con reloj.

Entradas			Salidas		Comentarios
S	R	CLK	Q	Q'	
0	0	X	Q_0	Q'_0	No Cambio
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	?	?	No válida

↑ = Transición del reloj de nivel BAJO a nivel ALTO

X = Irrelevante ("Condición indiferente")

Q_0 = Nivel de salida previo a la transición del reloj.

Instrucciones: Implementa el Flip-Flop R-S de la figura 8.10 y comprueba la tabla 8.4, la cual describe el comportamiento del circuito.

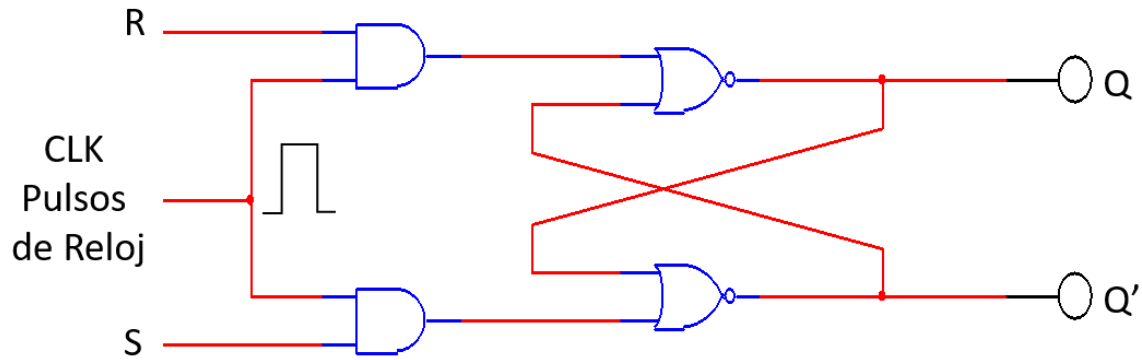


Figura 8. 12 Flip-Flop R-S con reloj.

Tabla 8.4 Tabla de la verdad extendida del Flip-Flop R-S con reloj.

FLIP-FLOP RS CON RELOJ			
Q	S	R	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	Indeterminado
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	Indeterminado

Práctica 8.5 Flip-Flop tipo D con disparo de Flanco ascendente.

Definición: El Flip-Flop tipo D resulta muy útil cuando se necesita almacenar un único bit de datos (1 o 0). Si se añade un inversor a un flip-flop S-R obtenemos un flip-flop D básico. Éste se dispara de igual manera en el flanco positivo. Por otro lado la entrada ya no será EN ahora en su lugar llevará un pulso de reloj lo que lo vuelve una entrada síncrona (CLK). Es la diferencia entre un LATCH y un Flip-Flop. En la *figura 8.13* se muestra el símbolo del Flip-Flop tipo D.

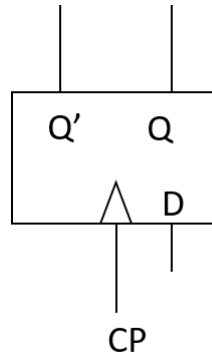


Figura 8. 13 Símbolo de un Flip-Flop tipo D.

El flip-flop D recibe su denominación debido a su capacidad de transferir “datos”. El inversor agregado reduce el número de entradas de dos a una. Este tipo de flip-flops algunas veces se denomina “un seguro-D con compuertas”.

La entrada CLK con frecuencia recibe la designación (de la inicial en inglés de compuerta, es decir gate) para indicar que esta entrada habilita el seguro con compuertas para hacer posible la entrada de información dentro del flip-flop. El "flip-flop" tipo D, sigue a la entrada, haciendo transiciones que coinciden con las de la entrada. El término "D", significa dato; este "flip-flop" almacena el valor que está en la línea de datos. Se puede considerar como una celda básica de memoria. La *tabla 8.5* muestra el comportamiento simplificado del Flip-Flop tipo D.

Tabla 8.5 Tabla de la verdad simplificada del Flip-Flop tipo D con reloj.

Entradas		Salidas		Comentarios
D	CLK	Q	Q'	
1	↑	1	0	SET (almacena un 1)
0	↑	0	1	RESET (Almacena un 0)

↑ = Transición del reloj de nivel BAJO a nivel ALTO

Instrucciones: Implementa el Flip-Flop tipo D de la figura 8.14 y comprueba la tabla 8.6, la cual describe el comportamiento del circuito.

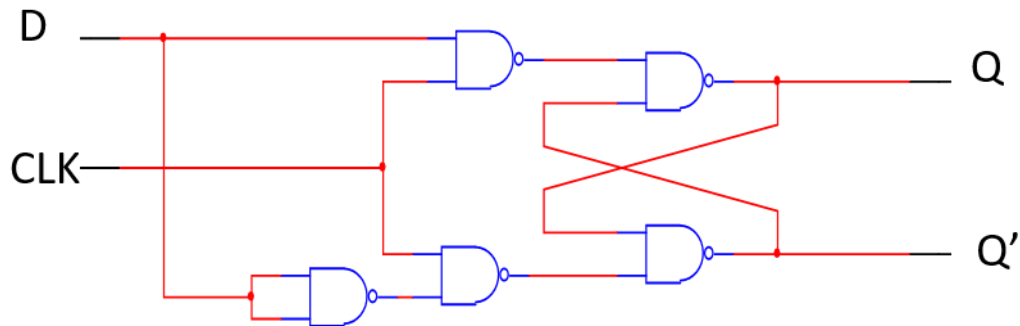


Figura 8. 14 Diagrama del Flip-Flop tipo D.

Tabla 8.5 Tabla de la verdad extendida del Flip-Flop tipo D con reloj.

FLIP-FLOP TIPO D		
Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Práctica 8.6 Flip-Flop tipo D con disparo de Flanco ascendente.

Definición: El flip-flop J-K es versátil y es uno de los tipos de flip-flop más ampliamente utilizado. El funcionamiento del flip-flop J-K es idéntico al del Flip-Flop S-R en las condiciones de operación SET, RESET y de permanencia de estado (no cambio). La diferencia está en que el flip-flop J-K no tiene condiciones no válidas como ocurre en el R-S. En la *figura 8.15* se muestra el símbolo del Flip-Flop J-K.

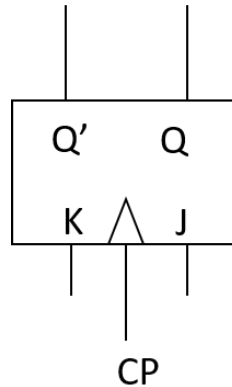


Figura 8. 15 Símbolo del Flip-Flop J-K.

En la *tabla 8.6* se muestra el comportamiento simplificado del Flip-Flop J-K.

Tabla 8.6 Tabla de la verdad simplificada del Flip-Flop J-K.

J	K	CLK	Q	Q'	Comentarios
0	0	↑	Q_0	Q'_0	No cambio
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	Q_0	Q'_0	Basculación

↑ = Transición del reloj de nivel BAJO a nivel ALTO
 Q_0 = Nivel de salida previo a la transición del reloj.

Instrucciones: Implementa el Flip-Flop J-K de la figura 8.16 y comprueba la tabla 8.7, la cual describe el comportamiento del circuito.

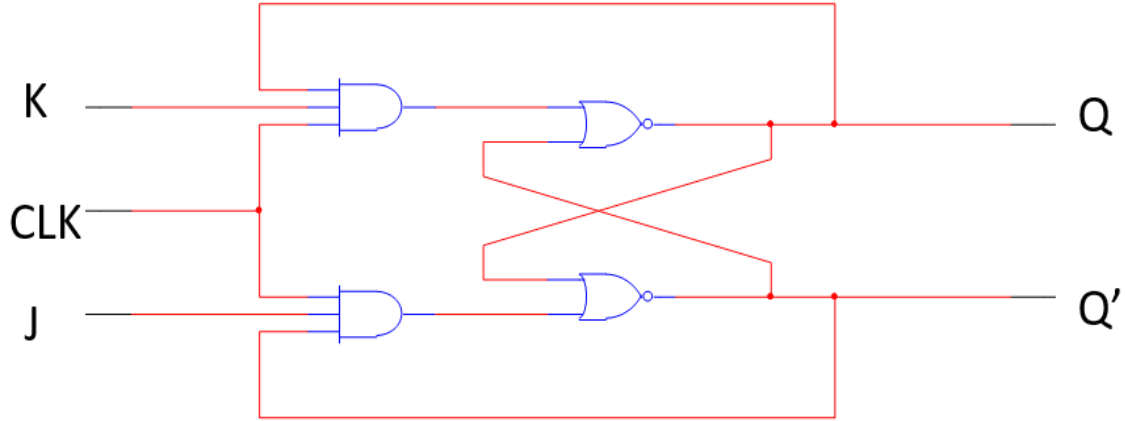


Figura 8. 16 Diagrama del Flip-Flop J-K

Tabla 8.7 Tabla de la verdad extendida del Flip-Flop J-K.

FLIP-FLOP JK CON RELOJ			
Q	J	K	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Práctica 8.7 Flip-flop tipo T con disparo de flanco descendente

Definición: El flip-flop T o "toggle" (conmutación) cambia la salida con cada borde descendente de pulso de clock, dando una salida que tiene la mitad de la frecuencia de la señal de entrada en T. El flip-flop tipo T es una versión de una sola entrada del flip-flop JK. El flip-flop tipo T se obtiene mediante un tipo JK si ambas entradas se ligan. La denominación T proviene de la capacidad del flip-flop para "conmutar" (de la inicial del termino en inglés: toggle), o cambiar de estado. En la *figura 8.17* se muestra el símbolo de un flip-flop tipo T.

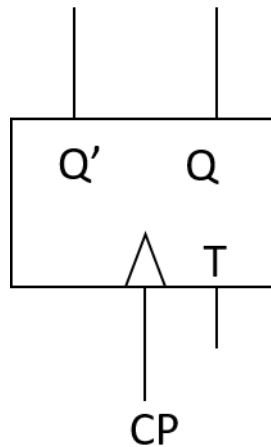


Figura 8. 17 Símbolo de un Flip-Flop tipo T.

Sin importar el estado presente del flip-flop, asume el estado complementario cuando ocurre el pulso de reloj mientras la entrada T es lógica 1. Es decir mientras $T=1$ el flip-flop estará conmutando y si $T=0$ estará sin cambios y guardará en memoria el estado actual.

Instrucciones: Implementa el Flip-Flop tipo T de la figura 8.18 y comprueba la tabla 8.8, la cual describe el comportamiento del circuito.

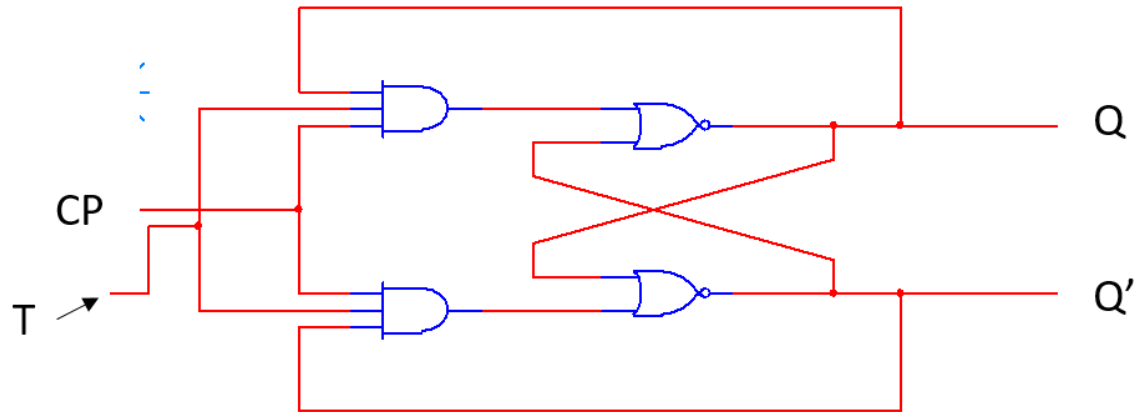


Figura 8. 18 Diagrama de un Flip-Flop tipo T.

Tabla 8.8 Tabla de la verdad del Flip-Flop tipo T.

Flip-flop tipo T		
Q	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Conclusiones Individuales
